

700105 Simulation & Concurrency



UNIVERSITY
of **HULL**

Coursework Questions

Do you have any questions about the coursework?

AI bonus lecture

AI in Games Roadmap

- Why AI in games is different
- Interacting with designers
 - Tools and processes
- Broad but shallow topics
 - Autonomous Agents in Free Space
 - World Representation & Path Finding
 - Decision Making Strategies

AI In Games

AI in Games

- There is an important distinction to be made between the AI studied by academics and that used in computer games. Academic research is split into two camps: *strong* AI and *weak* AI. The field of strong AI concerns itself with trying to create systems that mimic human thought processes and the field of weak AI with applying AI technologies to the solution of real-world problems. However, both of these fields tend to focus on solving a problem *optimally*, with less emphasis on hardware or time limitations. For example, some AI researchers are perfectly happy to leave a simulation running for hours, days, or even weeks on their 1000-processor Beowulf cluster so long as it has a happy ending they can write a paper about.

- Matt Buckland. *Programming Game AI by Example*

AI in Games

- Game AI programmers, on the other hand, have to work with limited resources. The amount of processor cycles and memory available varies from platform to platform but more often than not the AI guy will be left like Oliver holding out his bowl, begging for more. The upshot of this is that compromises often have to be made in order to get an acceptable level of performance.
 - Matt Buckland. *Programming Game AI by Example*
- “Some studios don’t do artificial intelligence – they just call it gameplay”

AI in Games

- In addition, successful games ... do one thing very well: They entertain the player Ipso facto, the AI must be entertaining, and to achieve this must more often than not be *designed to be suboptimal*. After all, most players will quickly become frustrated and despondent with an AI that always gives them a whippin'. To be enjoyable, an AI must put up a good fight, but will lose more often than win.
 - Matt Buckland. *Programming Game AI by Example*

Summary

- Programming for games is about running fast and being good enough and resources are allocated according to a pecking order
- Simulation
 - Should run at a frequency of at least ~60Hz and more likely ~200Hz
- Artificial intelligence is everything and nothing
 - Uses lots of well established Computer Science techniques, data structures and algorithms
 - Touches lots of other systems (locomotion, animation, audio etc.)
 - Artificial should be fun, and should pretend to be clever (but fair)

Considerations

- Time and space
 - How fast and how much memory are we using
- Environment representation
 - What do we know about the **virtual** world
- Optimizing for player fun
 - What will *“make the player feel clever, sly, cunning and powerful”*?
- The impact of immersion
 - If the AI does something that **might** be smart the player will project and speculate
 - Equally if the AI does something stupid (or impossible) immersion will be broken
- Programming for tool chains
 - The programmer writes code and the designer designs games
 - More iteration means a better game

Interacting with designers

Interacting with Designers

- AI is often designed by games designers who are implementing an overall vision
- We need to give them tools and interfaces into the game to give them the freedom they need without breaking the game
- We lean on well defined computer science concepts to allow them to do this
- We provide designers with safe ways to interact with the code so – often via scripting languages like Lua – to give them more power and flexibility at the expense of performance

Autonomous Agents in Free Space



Steering

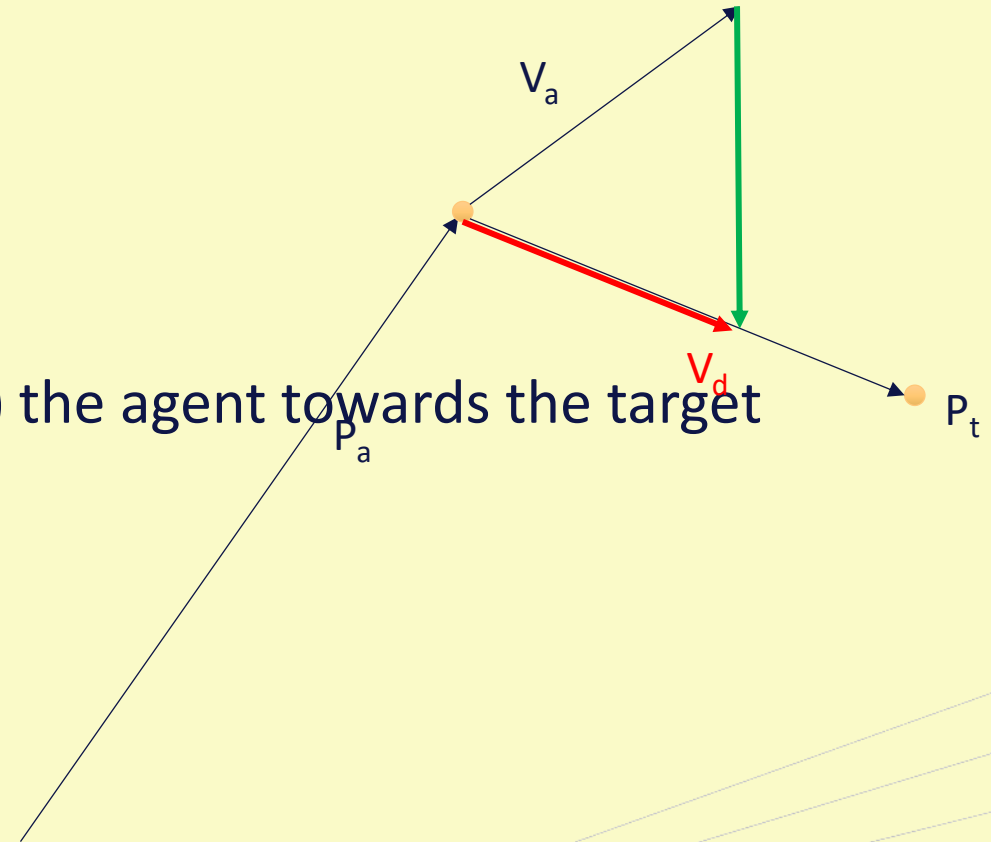
- Steering is basically picking a desired velocity and then matching that over time using simple physics

Steering Behaviours

- Seek
- Flee
- Arrive
- Follow Path
- Pursue
- Evade
- Wander
- Wall Avoidance
- Hide

Seek Behaviour

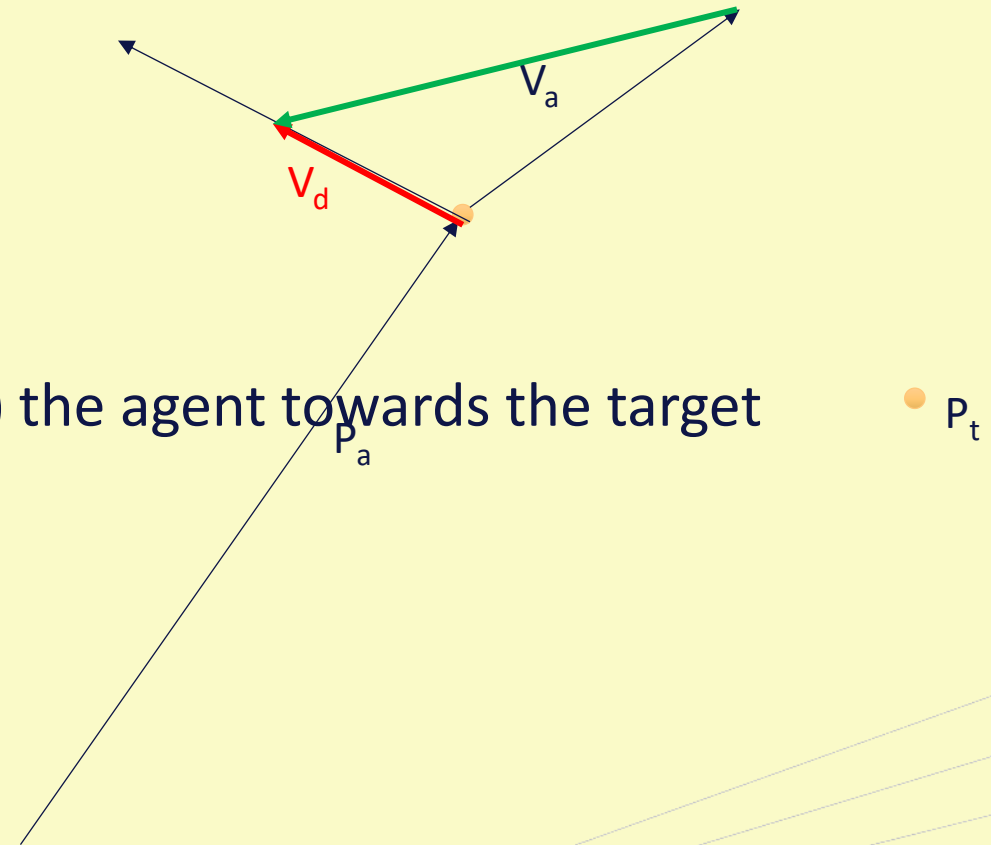
- Pick a target to seek
- Calculate the desired velocity V_d
 - $(P_t - P_a)$
 - Normalise
 - Multiply by maximum speed
- Calculate a **steering force** to push (accelerate) the agent towards the target
 - $V_d - V_a$





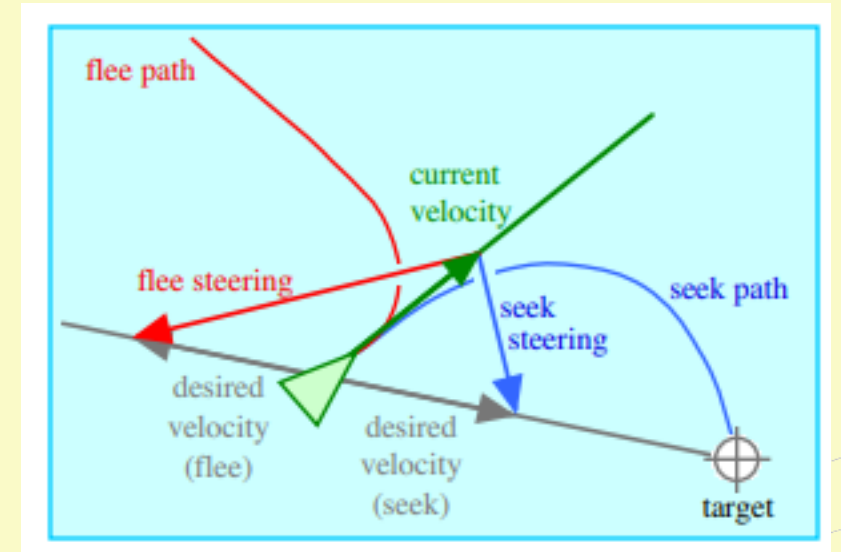
Flee Behaviour

- Pick a target to flee
- Calculate the desired velocity V_d
 - $(P_a - P_t)$
 - Normalise
 - Multiply by maximum speed
- Calculate a **steering force** to push (accelerate) the agent towards the target
 - $V_d - V_a$



Seek and Flee

- The green triangle is the agent
- The red line is the flee path from the target
- The blue line is the seek path to the target





Pursue Behaviour

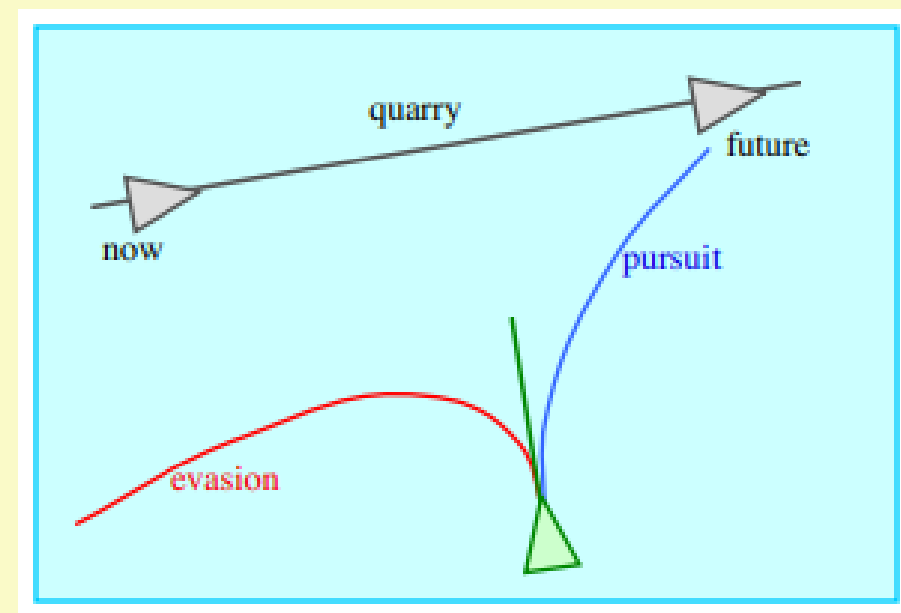
- Pick a ***moving*** target to pursue
- Calculate where it will be in a short amount of time
- Calculate the desired velocity V_d
 - $(P_t - P_a)$
 - Normalise
 - Multiply by maximum speed
- Calculate a **steering force** to push (accelerate) the agent towards the target
 - $V_d - V_a$

Evade Behaviour

- Pick a ***moving*** target to flee
- Calculate where it will be in a short amount of time
- Calculate the desired velocity V_d
 - $(P_t - P_a)$
 - Normalise
 - Multiply by maximum speed
- Calculate a **steering force** to push (accelerate) the agent towards the target
 - $V_d - V_a$

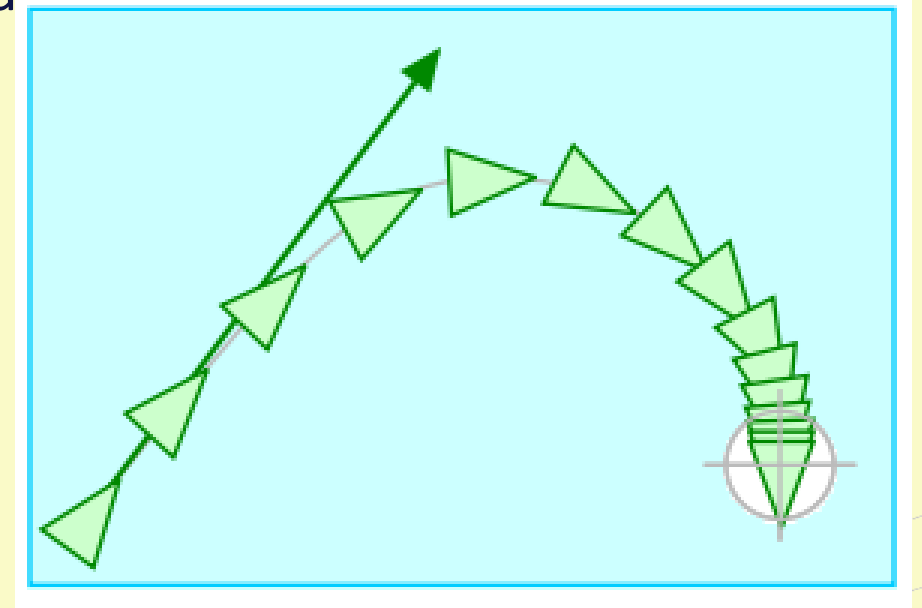
Pursue and Evade

- The green triangle is the agent
- The red line is the evade path from the target
- The blue line is the pursue path to the target



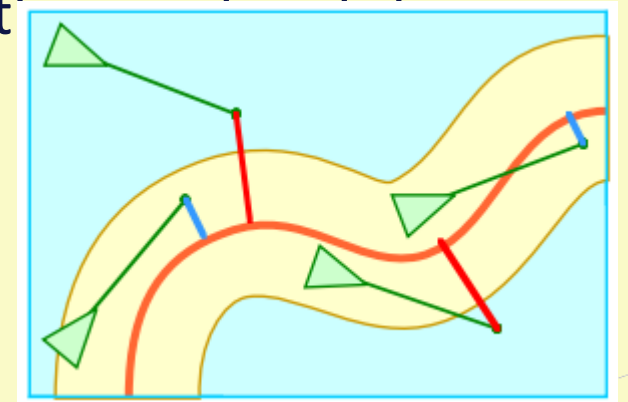
Arrive Behaviour

- Seek behaviour will overshoot it's target and then oscillate back and forth
- Arrive behaviour is the same as seek behaviour at a distance, but reduces desired velocity close to the target



Path Following

- In games a path is often created from a series of points in the world
- Following a path could be as simple as creating a series of arrive behaviours for each point on the path
- Or it could be more sophisticated, projecting the future location of the object and moving them gently around the path



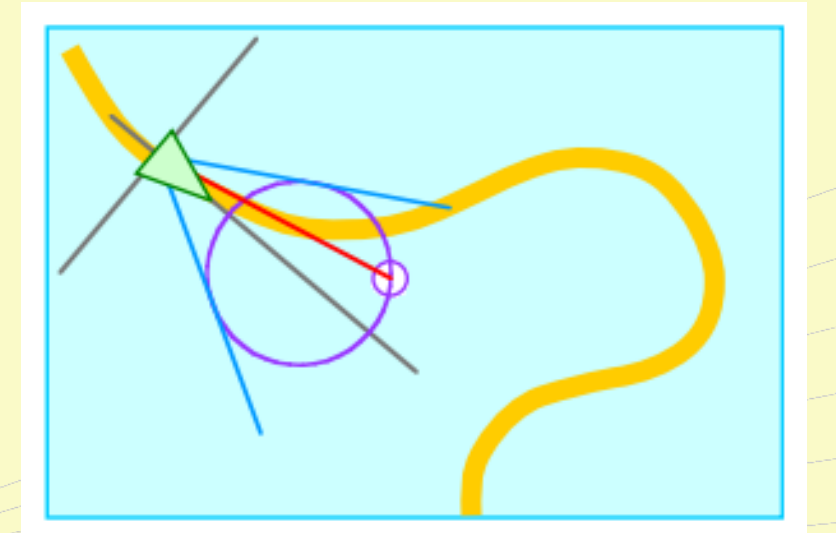
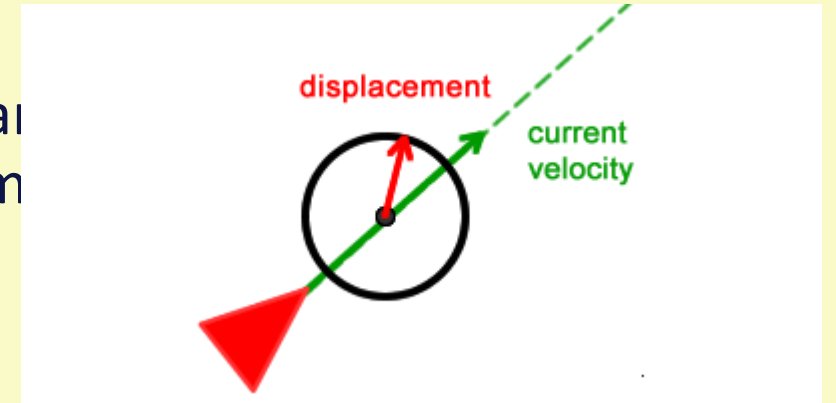


Follow Path

- Artificial Intelligence usually works with a simplified view of the virtual world
- Usually the world is represented as a **graph** based structure called a **Navigation Mesh** that is generated from the virtual world
- For more on Navigation Meshes see Cui, X. and Shi, H., 2011. A*-based pathfinding in modern computer games. *International Journal of Computer Science and Network Security*, 11(1), pp.125-130.
- This can be used to create a path of points through the virtual world

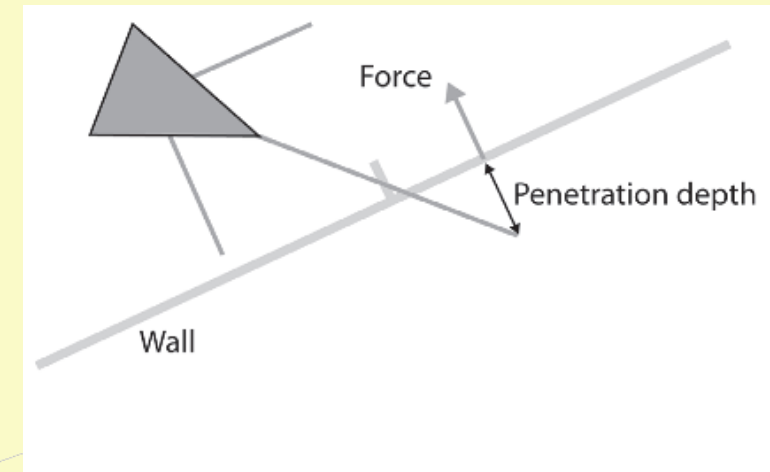
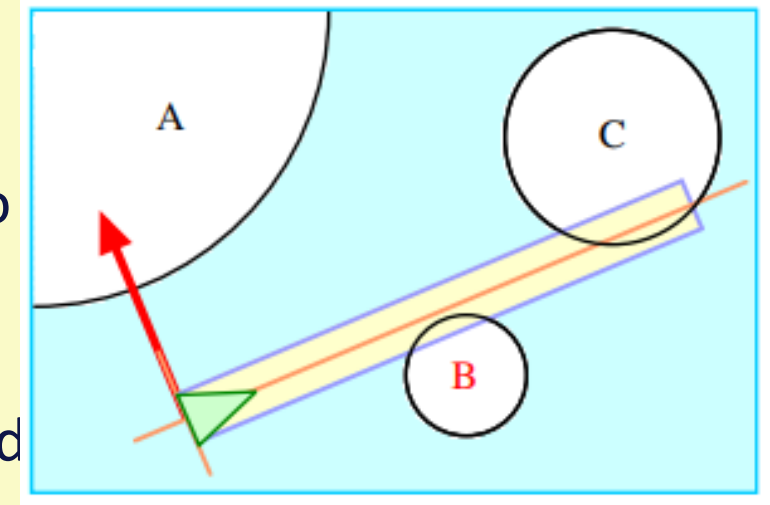
Wander Behaviour

- The wander behaviour gives an interesting idle movement
- Wandering involves projecting a circle ahead of the agent and the edge of the circle – displacing that point a little over time



Wall Avoidance

- Wall avoidance requires a (simplified) model of the world
- Agents use some method to detect if they may be about to
 - Swept shape
 - Whiskers
- If the agent is close to a wall they can use the penetration d





Hide Behaviour

- Hide behaviour can use the (simplified) world model to find a point where there is no line of sight between the point and the thing the agent wants to hide from
- These behaviours are getting more layered and game specific
- Hide involves finding a point to hide, then finding a path to that point, then moving to that point

Additional Resources

- Craig Reynolds GDC paper is on canvas and online at <https://www.red3d.com/cwr/steer/>
- Fernando Bevilacqua provides a nice set of short tutorials on steering at [Steering Tutorials by Envato Tuts+ \(tutsplus.com\)](https://www.tutsplus.com/articles/steering-tutorials)

Combining behaviours

- We've identified a few behaviours, but there are many more
- How can we achieve richer behaviour by combining multiple behaviours together

Steering Behaviours

- Seek
- Flee
- Arrive
- Pursue
- Evade
- Wander
- Separation
- Alignment
- Cohesion

Compound Behaviours

- Behaviour base class
 - pointers to the agent is belongs to for agent information (like position and velocity)
 - pointer to the world to get other important information (like other agents, obstacles etc.)
 - Virtual GetSteeringForce method
- We derive other behaviours from this class like SeekBehaviour or FleeBehaviour
- We can also create new behaviours that include and combine multiple other behaviours to build more complex behaviours

Priority Arbitration

- Only one behaviour executed at any time
- Behaviours are prioritized dynamically
- If a behaviour results a non-zero steering force the next highest priority behaviour is executed

Pros	Cons
Simple	Can only execute one behaviour at a time
Fast	

Weighted Blending / Weighted Sum

- All behaviours are considered
- Fixed set of weightings are applied
- Often used in flocking
- Potential to modify weights dynamically
- Can also perform a **Weighted Truncated Sum** (i.e. limit max force)

Pros	Cons
Simple(ish)	Can only execute one behaviour at a time
	Sometimes slow – every force is calculated every frame
	Behaviours can conflict and result in odd things happening (e.g. if a seek target is directly behind a flee target with similar weightings)
	Has potential to create a very large force (unless you truncate it)

Weighted Prioritised Truncated Sum

- Each behaviour is given a priority and a weight
- In order of priority steering forces are weighted and summed together until the accumulated steering force reaches some maximum, then truncate the remaining force

Pros	Cons
Simple(ish)	Can only execute one behaviour at a time
	Sometimes slow – every force is calculated every frame
	Behaviours can conflict and result in odd things happening (e.g. if a seek target is directly behind a flee target with similar weightings)
	Has potential to create a very large force (unless you truncate it)

Prioritised Dithering

- This was Craig Reynold's suggested strategy for combining flocking behaviours in his seminal flocking paper
- Each behaviour has a priority and a probability (0.0-1.0) of being implemented
- Generate a random number (0.0-1.0)
- If that is higher than the probability of the highest priority behaviour and that behaviour results in a non-zero steering force then apply that behaviour
- If not generate a new random number and move to the next highest priority behaviour

Pros	Cons
Simple(ish)	Can only execute one behaviour at a time
Fast	Unpredictable – requires a lot of tweaking



UNIVERSITY
OF HULL

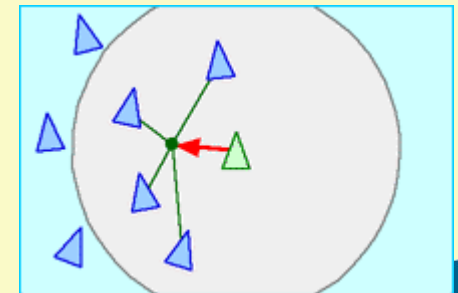
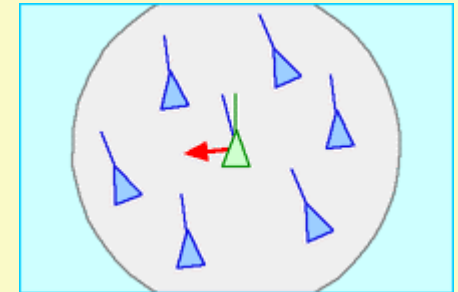
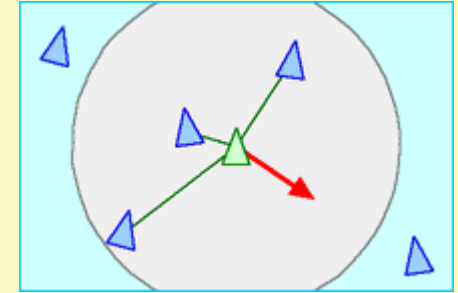
Flocking

Flocking

- *“The aggregate motion of a flock of birds, a herd of land animals, or a school of fish is a beautiful and familiar part of the natural world.”* (Reynolds, 1987)
- Penguins and Bats in Batman Returns (1992)
- Wildebeest in Lion King (1994)

Flocking

- Seminal work by Craig Reynolds (1987)
<https://www.red3d.com/cwr/papers/1987/boids.html>
- Flocking is a behaviour that takes into account other agents in the immediate neighbourhood
- Three new Steering behaviours
 - Separation is a steering force that moves an agent away from surrounding agents to ensure no collisions
 - Alignment is a steering force that aligns an agent's velocity with the agents around it
 - Cohesion is a steering force that moves an agent towards the average position of neighbouring agents
- There is a good description at
<https://www.red3d.com/cwr/boids/>





Things to consider

- There is lots of scope for experimentation
- State Buffering
 - Alignment is dependent on the velocities of the surrounding agents. If you update the velocity of one agent that will have a small impact on the next agent's update in the same frame.
 - If you want to avoid this you can do so by buffering state from frame to frame. For example, you might modify code to calculate and store the steering force for each agents, and then apply the force in the next step

Things to consider

- Efficiency - flocking looks good with a lot of agents, but that means typically $O(n^2)$ efficiency
 - Avoid repeating calculations
 - E.g. When determining an agent's neighbourhood the distance from agent A to agent B is the same as the distance from agent B to agent A
 - Avoid slow square roots



```
for (int i = 0; i < m_Agents.size(); i++)  
{  
    for (int j = 0; j < m_Agents.size(); j++)  
    {  
        if (i != j)  
        {  
            // test to see if i is close to j  
        }  
    }  
}
```



```
for (int i = 0; i < m_Agents.size() - 1; i++)  
{  
    for (int j = i + 1; j < m_Agents.size(); j++)  
    {  
        // test to see if i is close to j  
        // AND if j is close to i  
    }  
}
```

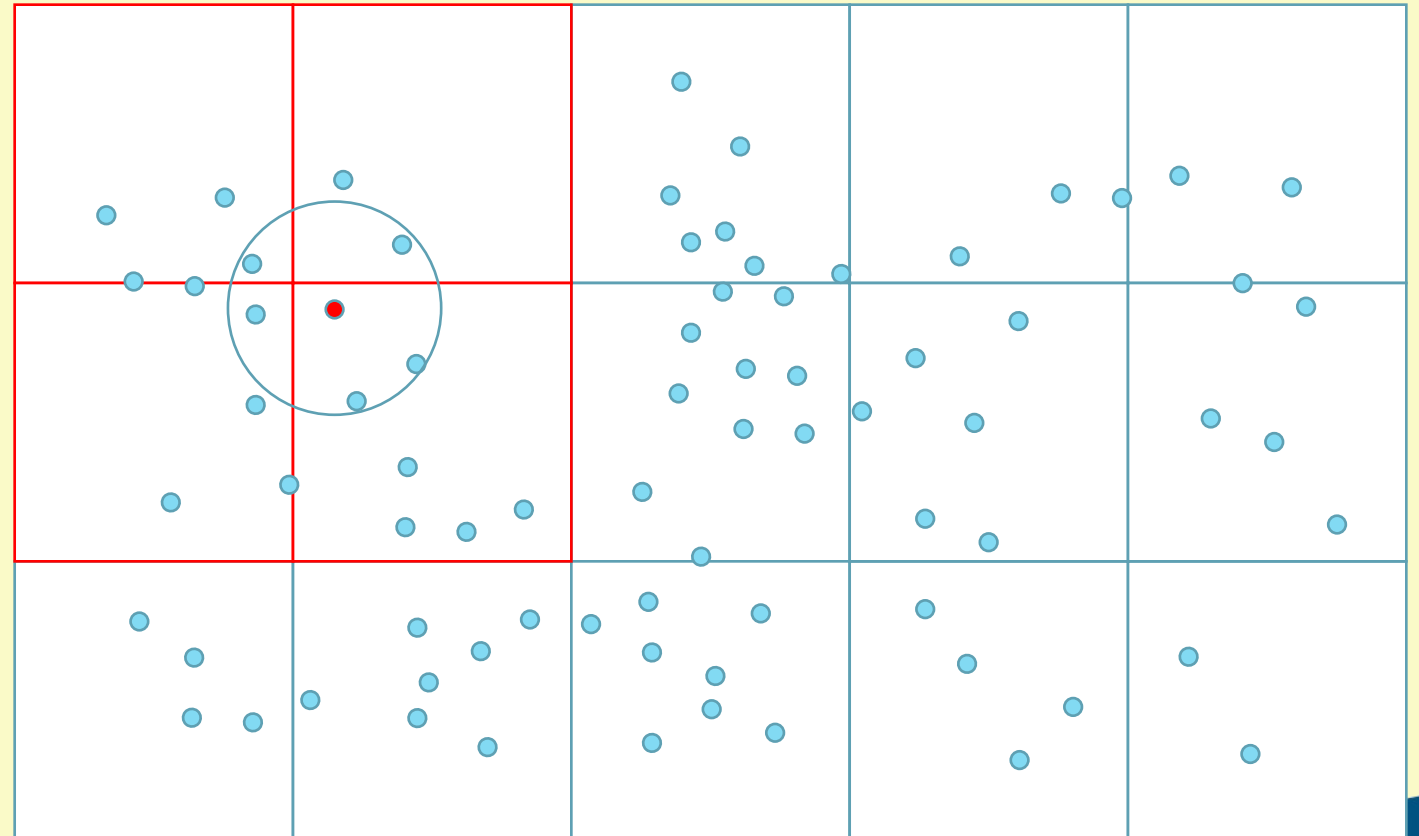


Things to consider

- Efficiency - flocking looks good with a lot of agents, but that means typically $O(n^2)$ efficiency
 - Avoid repeating calculations
 - E.g. When determining an agent's neighbourhood the distance from agent A to agent B is the same as the distance from agent B to agent A
 - Use fast maths calculations and avoid slow square roots
 - Implement spatial segmentation
 - Use a data structure to keep track of where things are in space
 - Octrees (or quadtrees) are okay, but simple flat grids are good too
 - Pay attention to agent size, and to max agent velocity

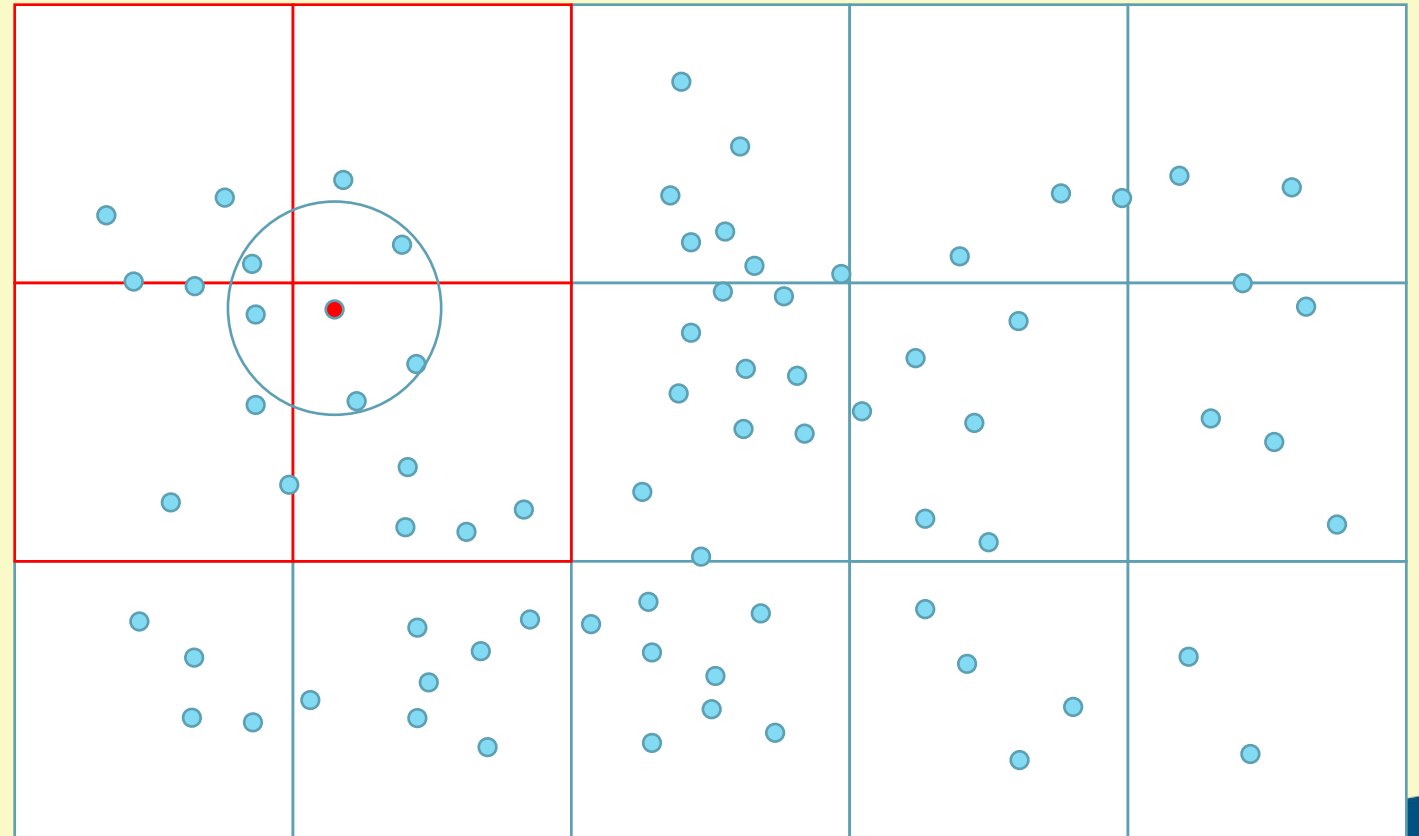
Grids

- Can limit the search for neighbours significantly
- Need neighbourhood radius to be smaller than grid size
- Calculate which corner the agent is in
- Test against agents in the same cell and three adjacent cells



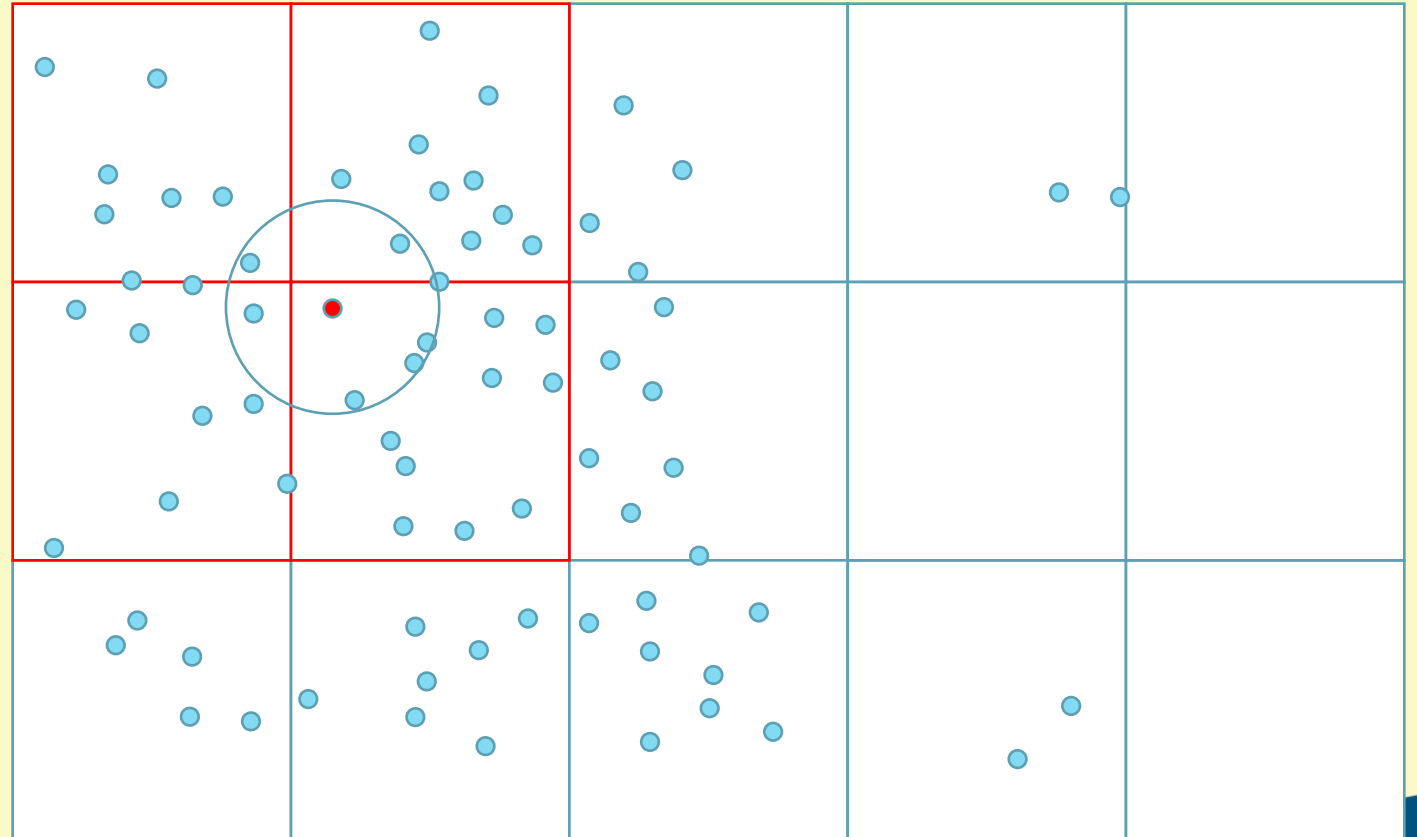
Grids

- In this example we test against just 17 other agents (instead of ~75) to find the neighbourhood of just 5
- Often structure like this will exist for different purposes (for rendering, or collision geometry) in a game and could be re-used



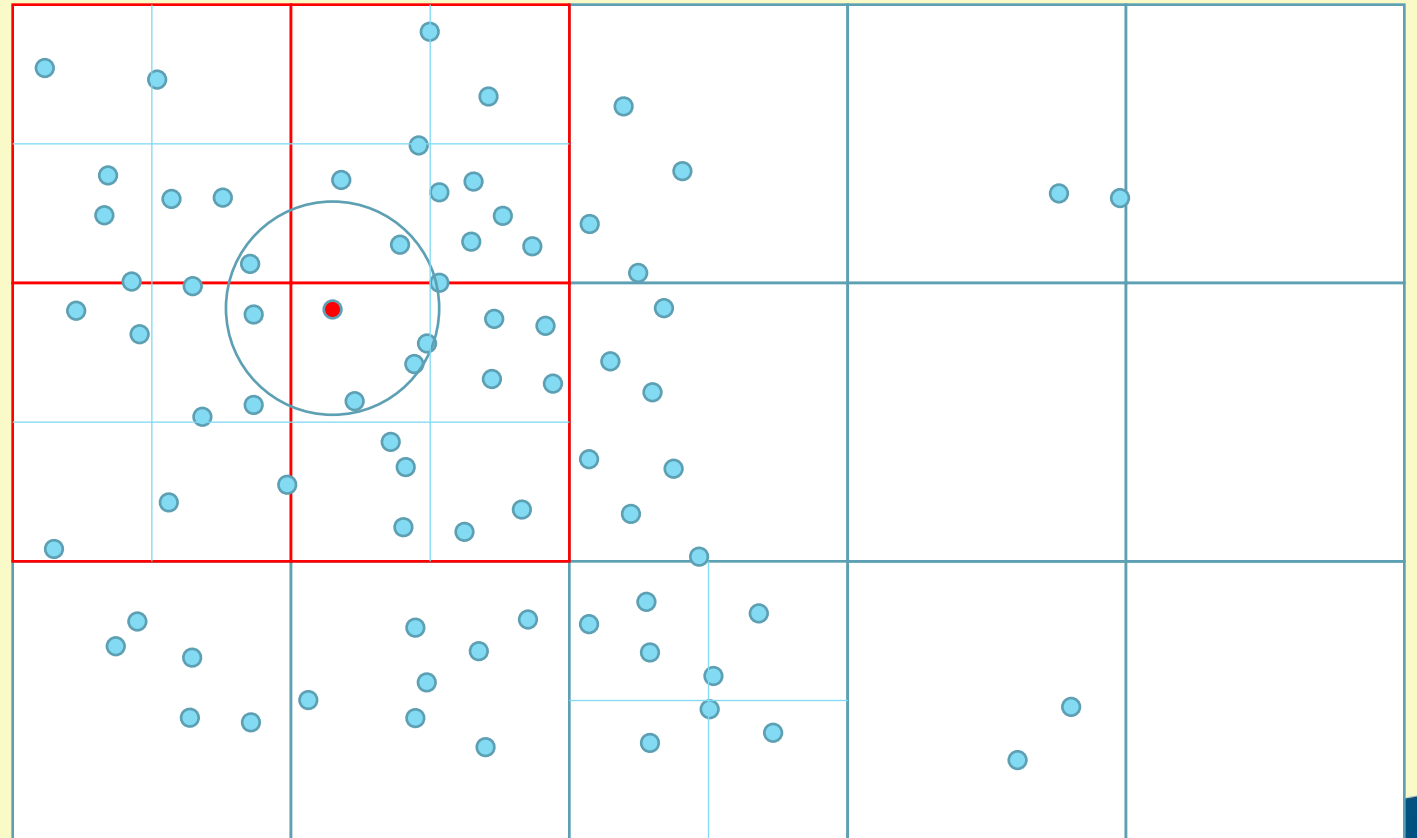
Problem

- In this example we test against just 17 other agents (instead of ~75) to find the neighbourhood of just 5
- Often structure like this will exist for different purposes (for rendering, or collision geometry) in a game and could be re-used



Quadtree

- In this example we test against just 17 other agents (instead of ~75) to find the neighbourhood of just 5
- Often structure like this will exist for different purposes (for rendering, or collision geometry) in a game and could be re-used



Additional Resource

- Flocking coding challenge <https://www.youtube.com/watch?v=mhjuuHl6qHM&t=2s>
- Reynold's 1987 Siggraph Paper <https://www.red3d.com/cwr/papers/1987/boids.html>



Summary

- Lots of questions, but the answer is usually “suck it and see”
- Suggests a flexible and dynamic (and probably over-engineered) approach
- The important thing is to recognise some of the problems and their solutions
 - Conflicting forces -> assign priorities
 - Steering force too large -> truncate it
 - Need to ensure something never happens -> hack it
 - For example if two agents end up somewhere they shouldn't you can just nudge them back to where they should be
 - If the scene is very crowded though this can cause further problems

World Representation & Path Finding

World Representation

- Highly game dependent
- Mechanically significant
 - Number of dimensions
 - Discrete or continuous space
 - Flat or nested space
- Often we need to simplify the world

Chess example

- Discrete space
- 2D
- Flat



Pac-Man Example

- Continuous space
- 2D
- Flat



Doom Example

- Continuous
- 2D ?
- Flat



Zelda Example

- Continuous
- 2D
- Nested



Blackboards

- Often we can share information with other systems
- Often we can share information using a blackboard pattern
 - Effectively a global source of information
 - May cause issues concerning what individual agents should know
- https://en.wikipedia.org/wiki/Blackboard_system



UNIVERSITY
OF HULL

NavMeshes

- Start with the level
- Create a series of convex walkable polygons
- Can be used to build a graph





UNIVERSITY
OF HULL

Graphs

- A Graph is a series of nodes linked to each other by edges
- Graphs are common in computer science
 - Trees are a special type of graphs
- Graph creation become a part of the tool chain
 - Either automated, manual or hybrid



Graph Usage

- Most often we talk about graphs with respect to navigation
- Graphs can be used to represent lots of other things
 - Dependencies in resource management games or skill trees
 - (if I need Z then first I need to do X and Y)
 - State space
 - If I do X then they do Y then I can do Z
 - A Finite State Machine could be described as a graph

Nodes and Edges

- Usually nodes represent values and edges represent moving between values
- In some graphs edges are bidirectional (digraphs)
- Sometimes edges have weights
 - Maybe the cost to perform an action
 - Maybe the time taken to move from one node to another
- Sometimes edges have different weights depending on the direction of travel
 - Walking up a hill is more difficult than walking down a hill

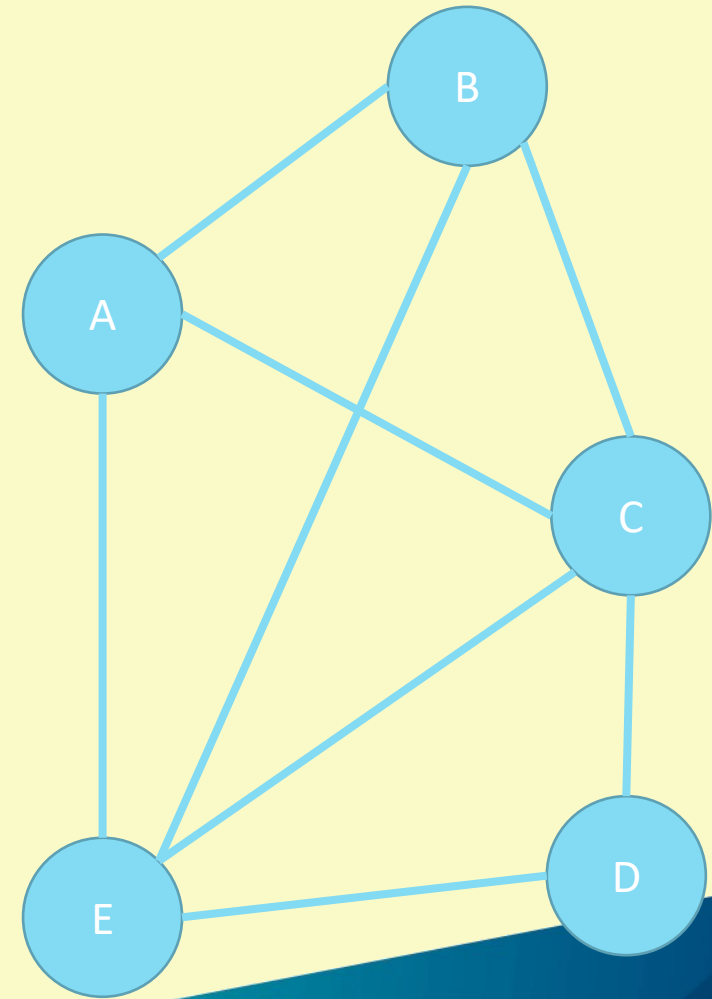
Highly game dependent

- There are no standard implementations of graphs (like there are for lists, for example)
- Binary trees are a special case of a graph
- Grids are a special case of a graph
 - The implementation and algorithms used can take advantage of this
- There is no one right way – the answer is always *it depends*
 - You need to understand enough to recognise where
 - For example, usually A* search is the best option, however in a uniformly weighted grid Jump Point Search outperforms A* search

Graph Representation – Adjacency Matrix

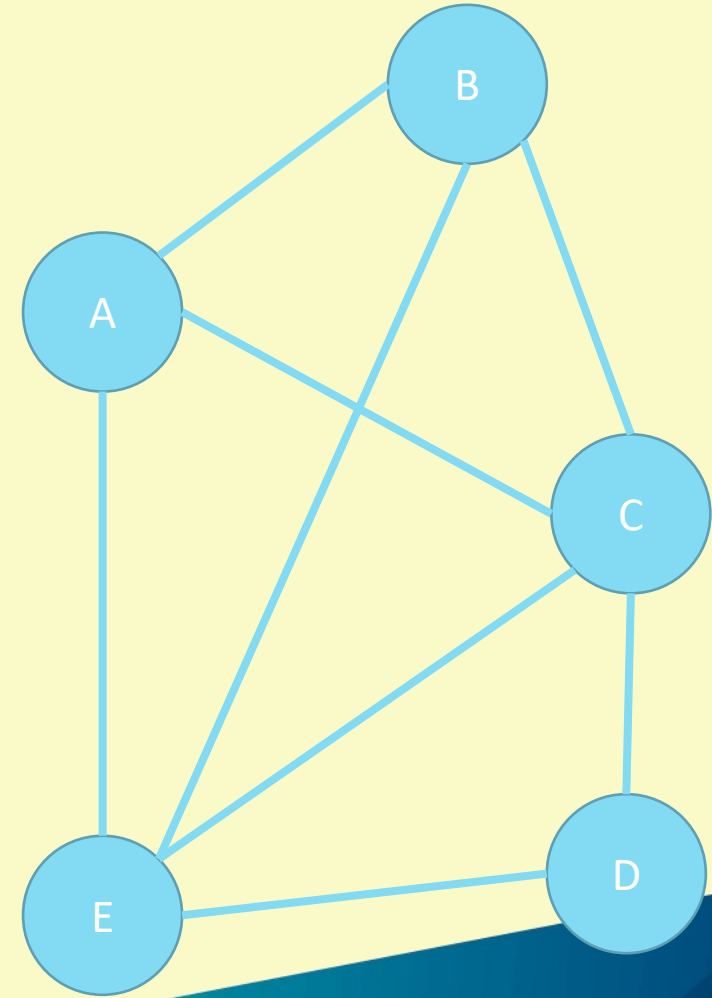
- If the graph is **dense** then we can use an adjacency matrix
 - “dense” means most nodes are linked to most other nodes.
 - Values could represent a weight

	A	B	C	D	E
A	0	1	1	0	1
B	1	0	1	0	1
C	1	1	0	1	1
D	0	0	1	0	1
E	1	1	1	1	0



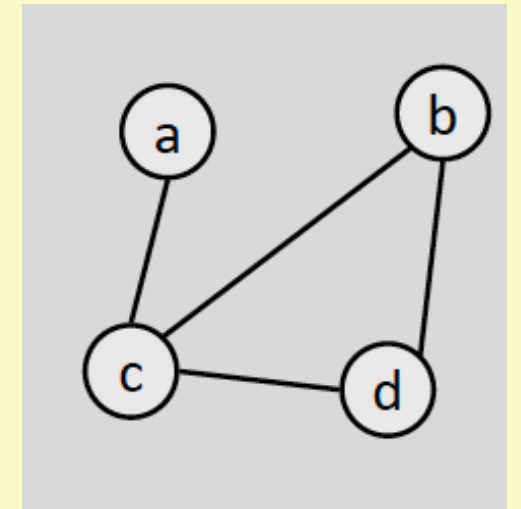
Graph Representation – Adjacency List

- If the graph is **sparse** then we can use an adjacency list
 - “sparse” means most nodes are only linked to a small number of other nodes
 - This is more typical in games
 - A → B, C, E
 - B → A, C, E
 - C → A, B, D, E
 - D → C, D



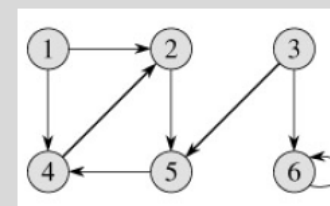
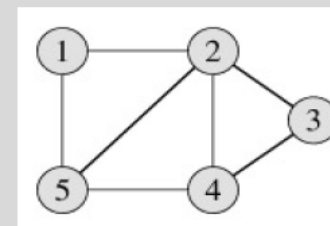
Graphs definitions

- **Graph** is a data structure:
 - a set of **vertices** V , (or nodes)
 - a set of **edges** E , where an edge represents a connection between 2 vertices.
 - Graph $G = (V, E)$
 - edge is a pair (v, w) where v, w are in V
- For example, the graph at right,
 $V = \{a, b, c, d\}$
 $E = \{(a, c), (b, c), (b, d), (c, d)\}$
- **Degree**: number of edges touching a given vertex.
at right: $a=1, b=2, c=3, d=2$



Graph Terminology - overview

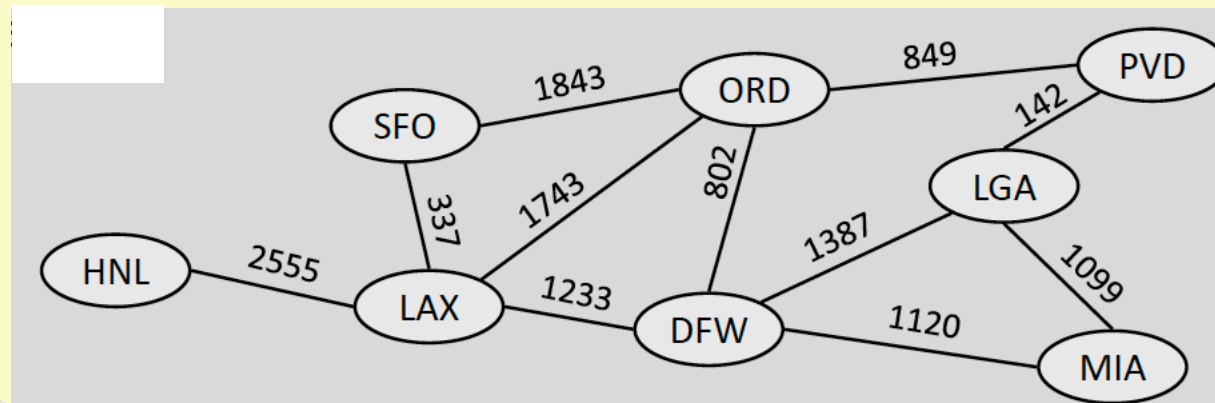
- Two vertices in a graph are **adjacent** if there is an edge connecting the vertices.
 - Example (1,2), (1,5)
- Two vertices are on a **path** if there is a sequences of vertices beginning with the first one and ending with the second one
- Graphs with ordered edges are **directed**. For directed graphs, vertices have **in and out degrees**.
- **Weighted** Graphs have values associated with edges.





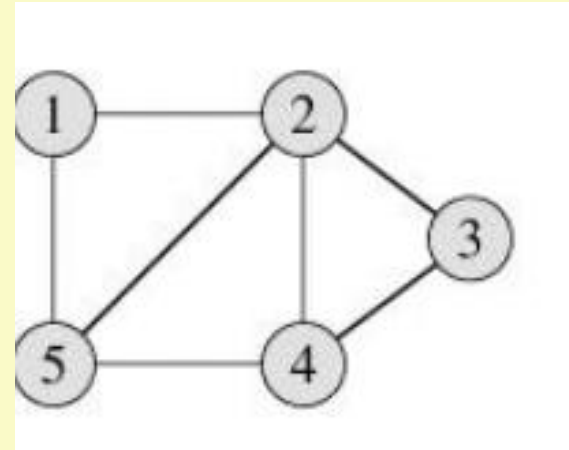
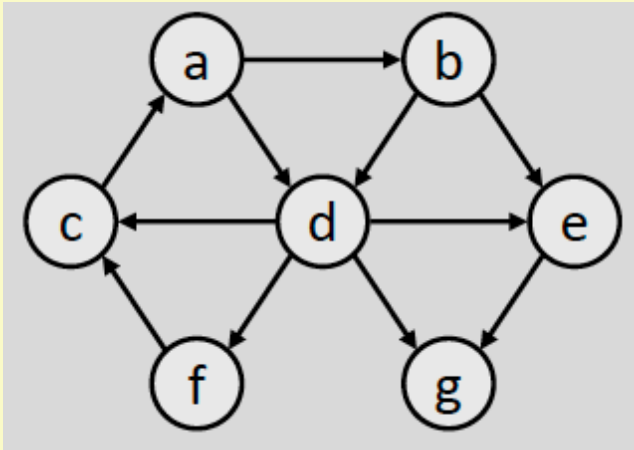
Weighted Graphs

- **weight:** Cost associated with a given edge.
 - Some graphs have weighted edges, and some are unweighted.
 - Edges in an unweighted graph can be thought of as having equal weight (e.g. all 0, or all 1, etc.)
 - Most graphs do not allow negative weights.
- *example:* graph of airline flights, weighted by miles between cities:



Directed vs undirected Graphs

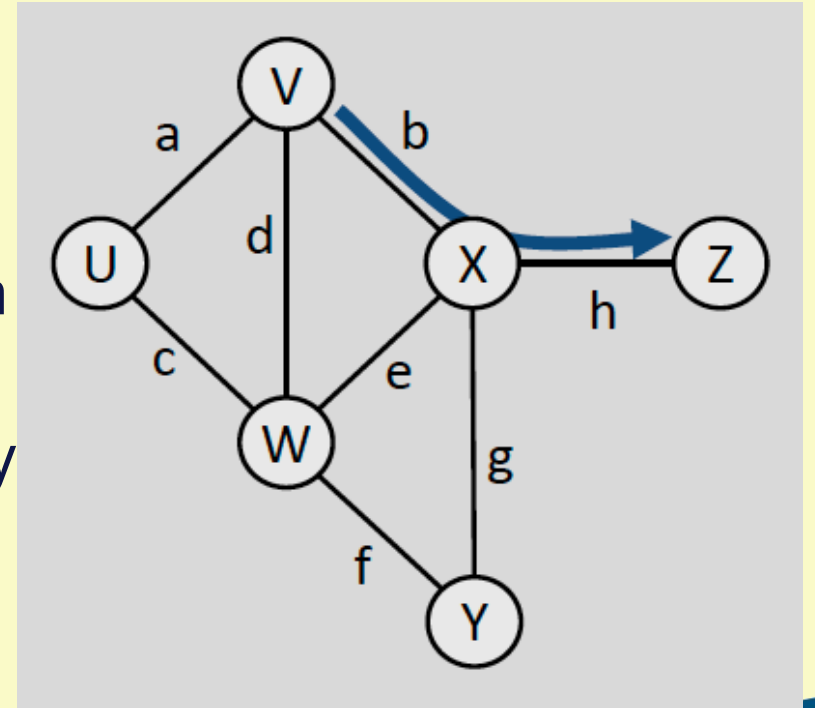
- **directed graph**("digraph"): One where edges are *one-way* connections between vertices.
- Undirected graph: the connections is bidirectional





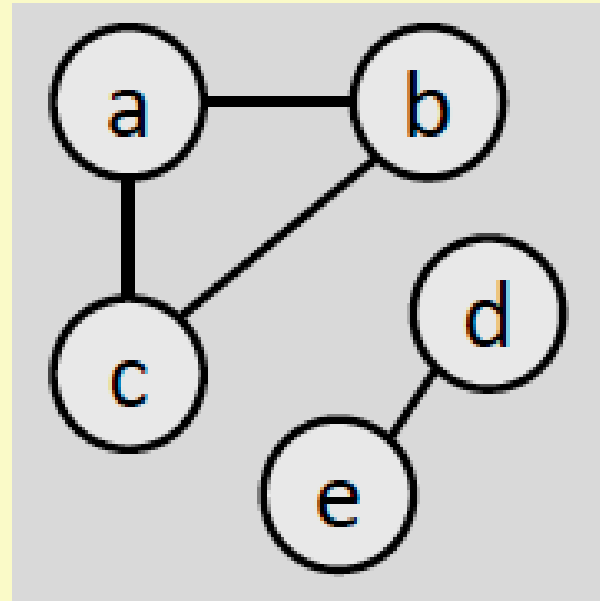
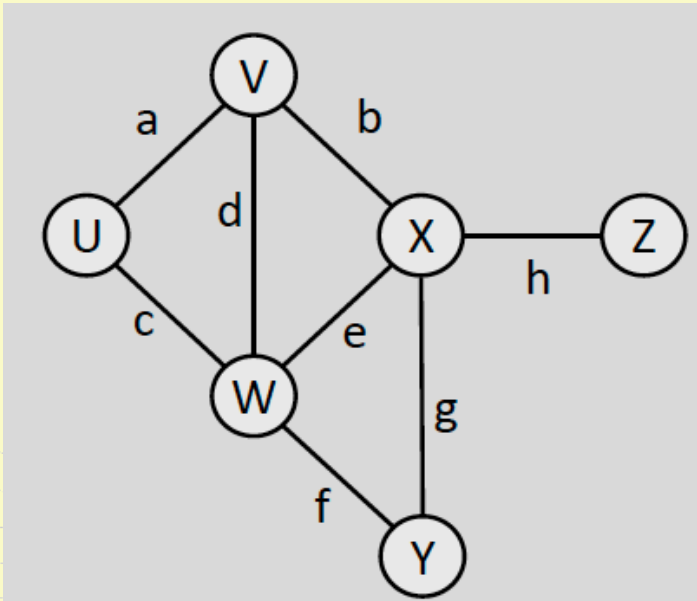
Paths

- **path:** A path from vertex a to b is a sequence of edges that can be followed starting from a to reach b .
 - can be represented as vertices visited, or edges taken
 - example, one path from V to Z : $\{b, h\}$ or $\{V, X, Z\}$
 - What are three paths from U to Y ?
- **path length:** Number of vertices or edges contained in the path.
- **Neighbor** or **adjacent:** Two vertices connected directly by an edge.
 - example: V and X



Reachability, Connectedness

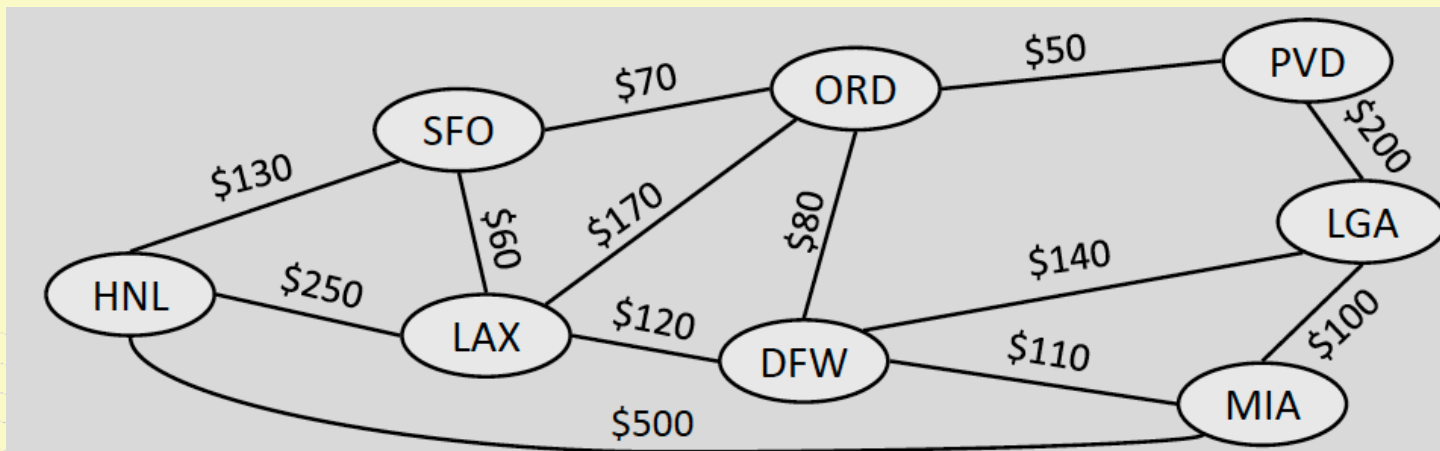
- **reachable:** Vertex a is *reachable* from b if a path exists from a to b .
- **connected:** A graph is *connected* if every vertex is reachable from any other.
 - Is the graph connected?





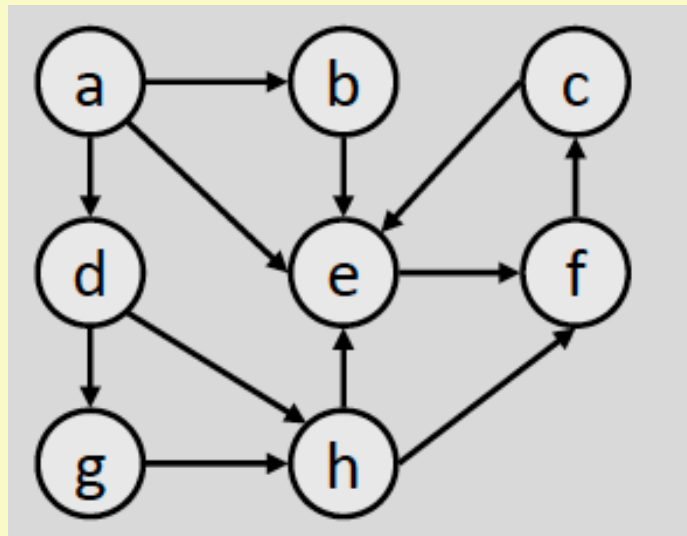
Searching for paths

- Searching for a path from one vertex to another:
 - Sometimes, we just want *any* path (or want to know there is a path).
 - Sometimes, we want to minimize path *length*(# of edges).
 - Sometimes, we want to minimize path *cost*(sum of edge weights).
- What is the shortest path from MIA to SFO?
- Which path has the minimum cost?



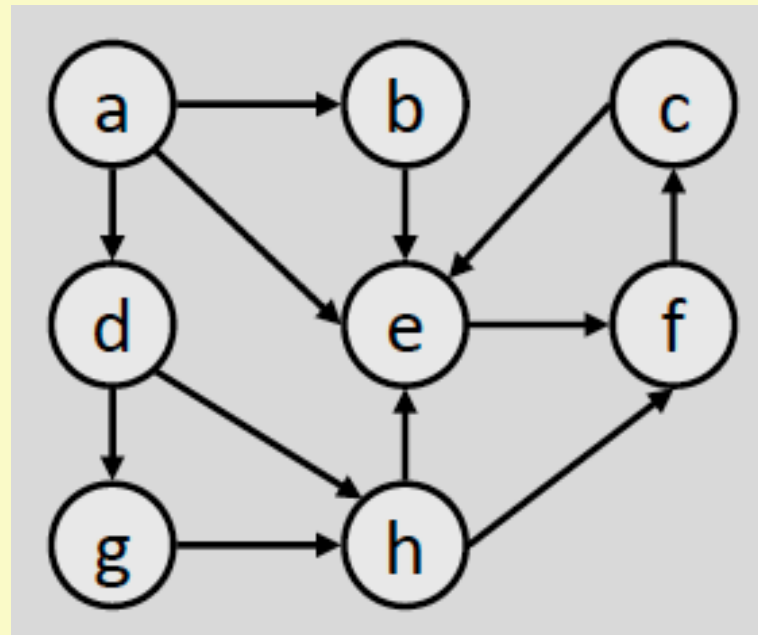
Depth-first Search

- **depth-first search(DFS):** Finds a path between two vertices by exploring each possible path as far as possible before backtracking.
 - It searches 'deeper' the graph when possible.
 - Starts at the selected node and explores as far as possible along each branch before backtracking.
- Depth-first paths from *a* to all vertices (assuming ABC edge order):
 - to b: {a, b}
 - to c: {a, b, e, f, c}
 - to d: {a, d}
 - to e: {a, b, e}
 - to f: {a, b, e, f}
 - to g: {a, d, g}
 - to h: {a, d, g, h}



Breadth-first Search

- **breadth-first search**(BFS): Finds a path between two nodes by taking one step down all paths and then immediately backtracking.
- BFS always returns the shortest path (the one with the fewest edges) between the start and the end vertices.
 - to b: {a, b}
 - to c: {a, e, f, c}
 - to d: {a, d}
 - to e: {a, e}
 - to f: {a, e, f}
 - to g: {a, d, g}
 - to h: {a, d, h}

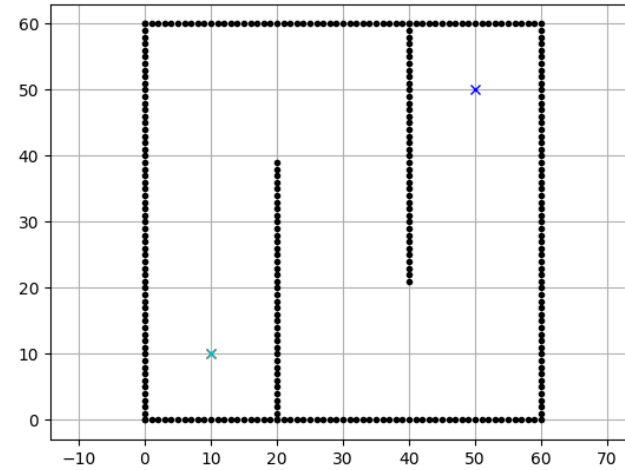




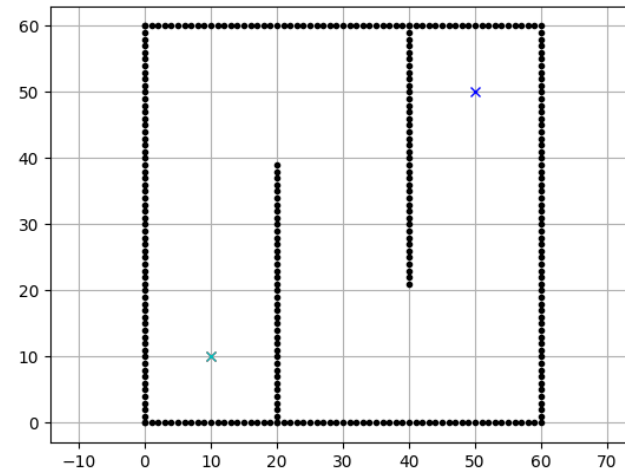
UNIVERSITY
OF HULL

Path planning at a glance

- Dijkstra
- A*
- Artificial Potential Field
- Roadmap approach
- Cell Decomposition
- ...



Dijkstra



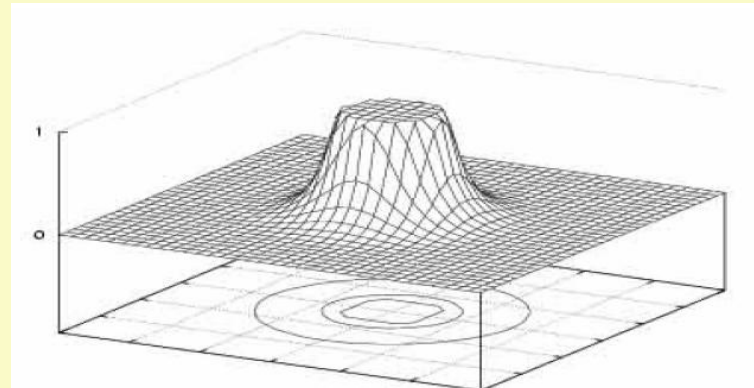
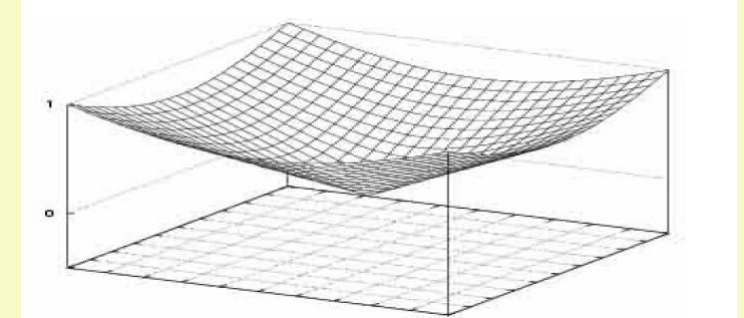
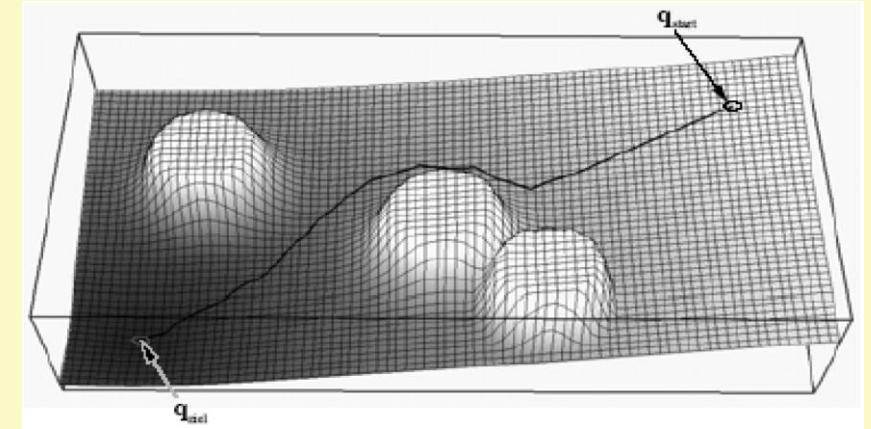
A*



UNIVERSITY
OF HULL

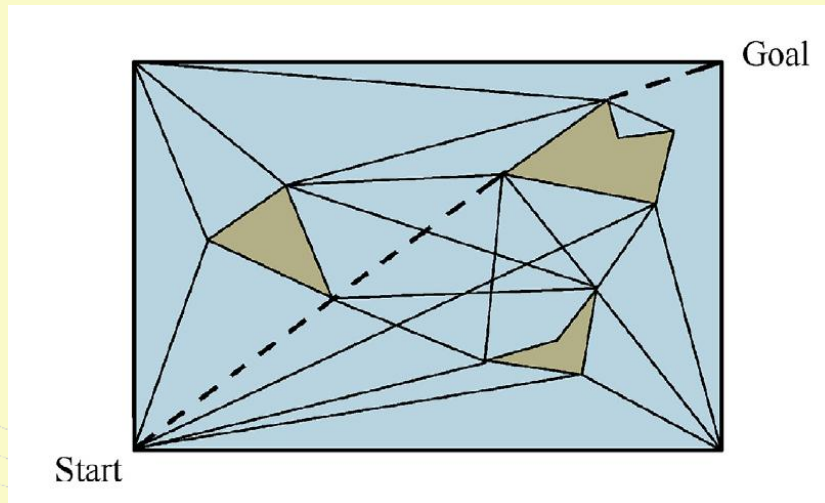
Artificial Potential Field

- Plot a path using attractive and repulsive forces
- Could create behaviours to do this

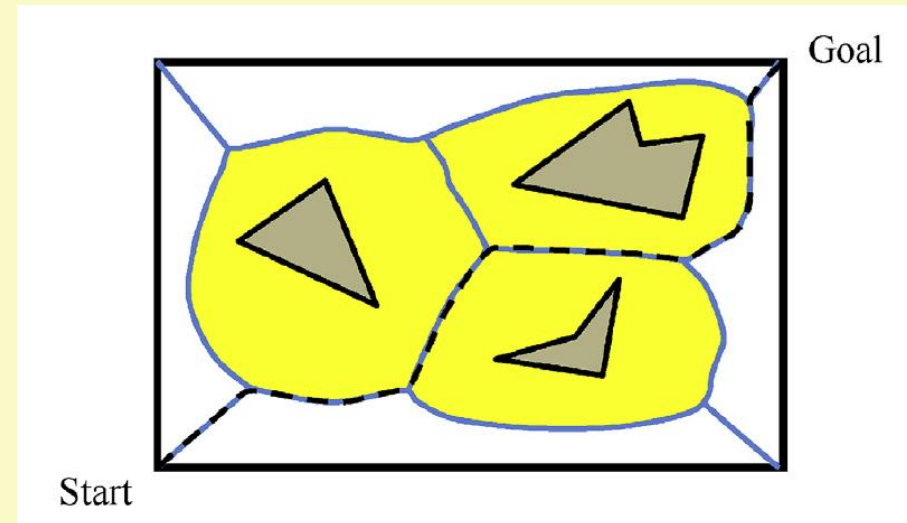


Roadmap approach

- The dark areas are obstacles
- The dashed lines are the path
- Not efficient to get the shortest path
- This could be the start of automatic generation of a nav mesh



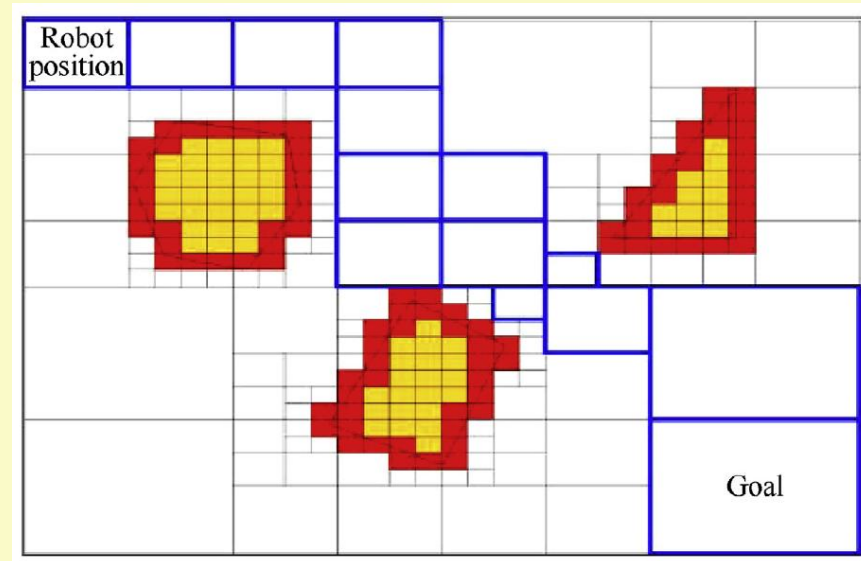
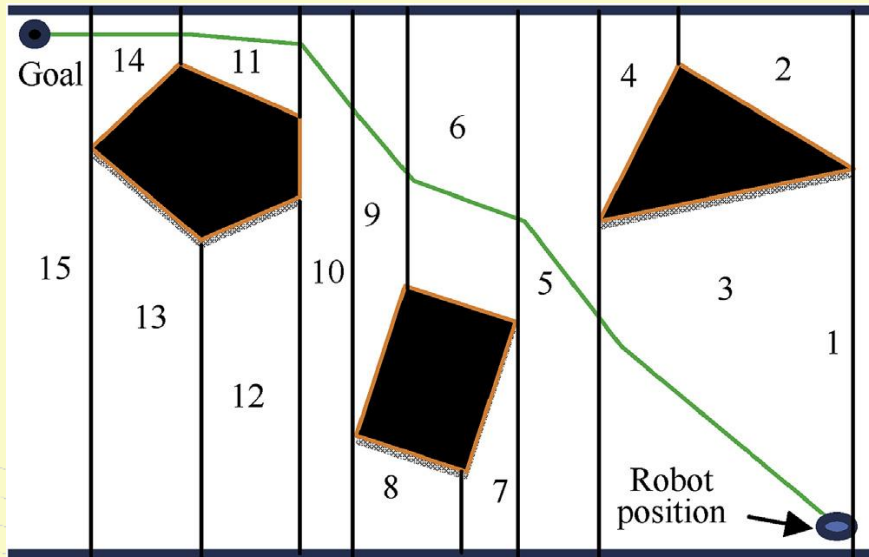
Visibility graph



Voronoi diagram

Cell Decomposition (CD)

- Non-overlapping grids
- Pure cells (free of obstacles)
- Corrupted cells (containing obstacles)
- The path is the sequence of pure cells that joined from start to goal.



Adaptive CD

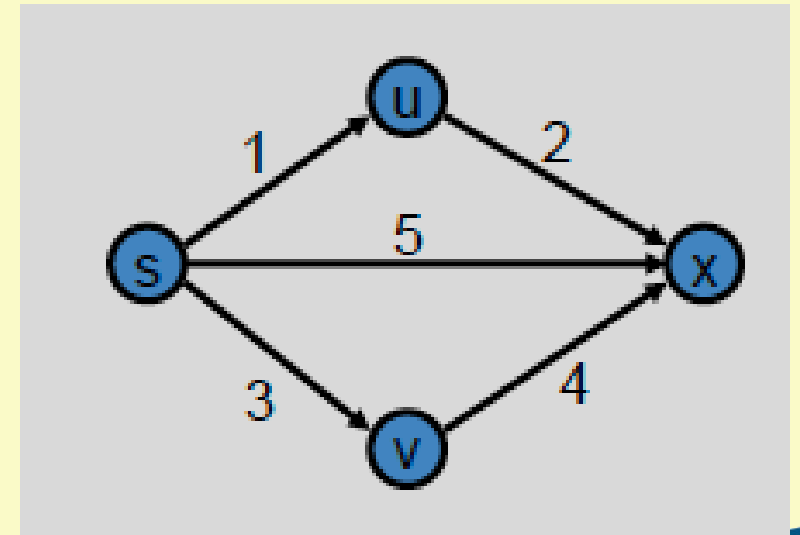


UNIVERSITY
OF HULL

Dijkstra's Algorithm

Dijkstra's Algorithm

- Single source shortest path problem
- Finds the shortest path tree from a specific start node to every other node
- Given a graph and a start vertex s
 - Determine distance of every vertex from s
 - Identify shortest paths to each vertex
 - Express concisely as a “shortest paths tree”
 - Each vertex has a pointer to a predecessor on shortest path



Shortest path algorithm: Dijkstra

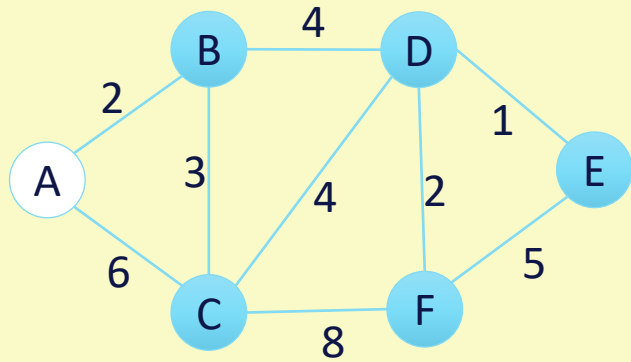
- **Dijkstra algorithm** is a solution to the single-source shortest path problem in graph theory.
- Works on both directed and undirected graphs. However, all edges must have nonnegative weights.
- Approach: Greedy
- Input: Weighted graph $G=\{E,V\}$ and source vertex $v \in V$, such that all edge weights are nonnegative
- Output: Lengths of shortest paths (or the shortest paths themselves) from a given source vertex $v \in V$ to all other vertices

Dijkstra algorithm

Dijkstra's Algorithm – Pseudocode

```
dist[s] ← 0                                (distance to source vertex is zero)
for all v ∈ V - {s}
    do dist[v] ← ∞                          (set all other distances to infinity)
S ← ∅                                        (S, the set of visited vertices is initially empty)
Q ← V                                       (Q, the queue initially contains all vertices)
while Q ≠ ∅                                (while the queue is not empty)
do u ← mindistance(Q, dist)                (select the element of Q with the min. distance)
    S ← S ∪ {u}                             (add u to list of visited vertices)
    for all v ∈ neighbors[u]
        do if dist[v] > dist[u] + w(u, v)    (if new shortest path found)
            then d[v] ← d[u] + w(u, v)      (set new value of shortest path)
            (if desired, add traceback code)
return dist
```

Dijkstra Example

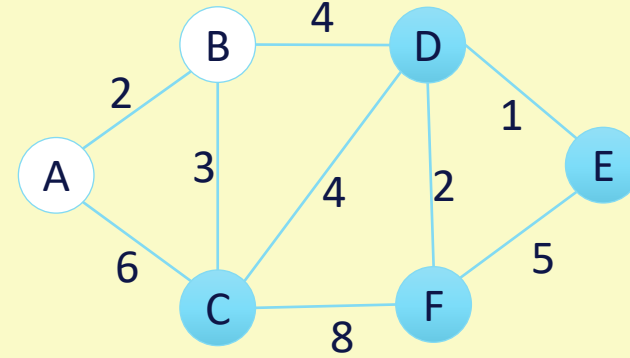
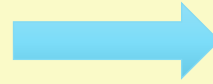
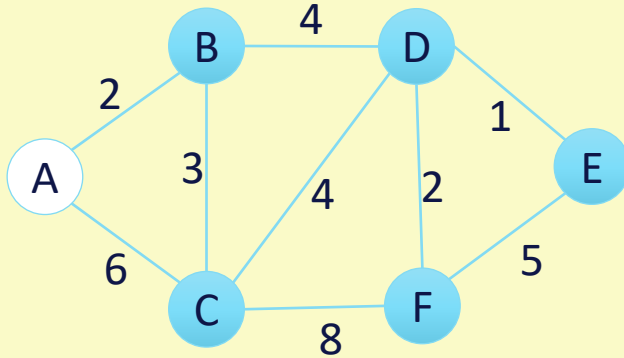


Source: A, Destination: F

$S=\{A\}$ $U=\{B,C,D,E,F\}$

	A	B	C	D	E	F
A	-	inf	inf	inf	inf	inf

Dijkstra Example



Source: A, Destination: F

$S=\{A\}$ $U=\{B,C,D,E,F\}$

	A	B	C	D	E	F
A	-	inf	inf	inf	inf	inf
	-	2(-)	6	inf	inf	inf

$A \rightarrow B = 2$

$S=\{A, B\}$ $U=\{C,D,E,F\}$

$A \rightarrow B = 2$

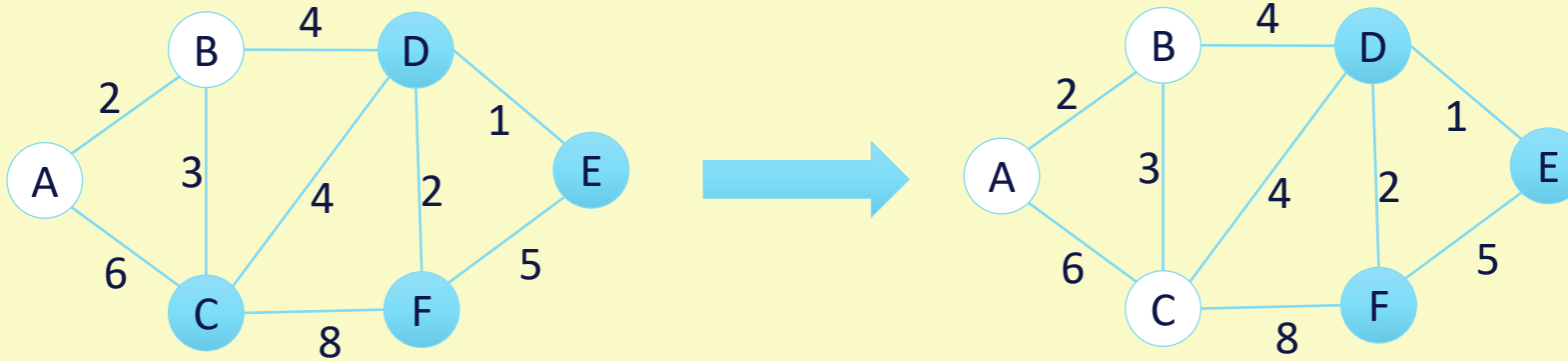
$A \rightarrow C = 6$

$A \rightarrow D = \text{inf}$

$A \rightarrow E = \text{inf}$

$A \rightarrow F = \text{inf}$

Dijkstra Example



Source: A, Destination: F

$S = \{A, B\}$ $U = \{C, D, E, F\}$

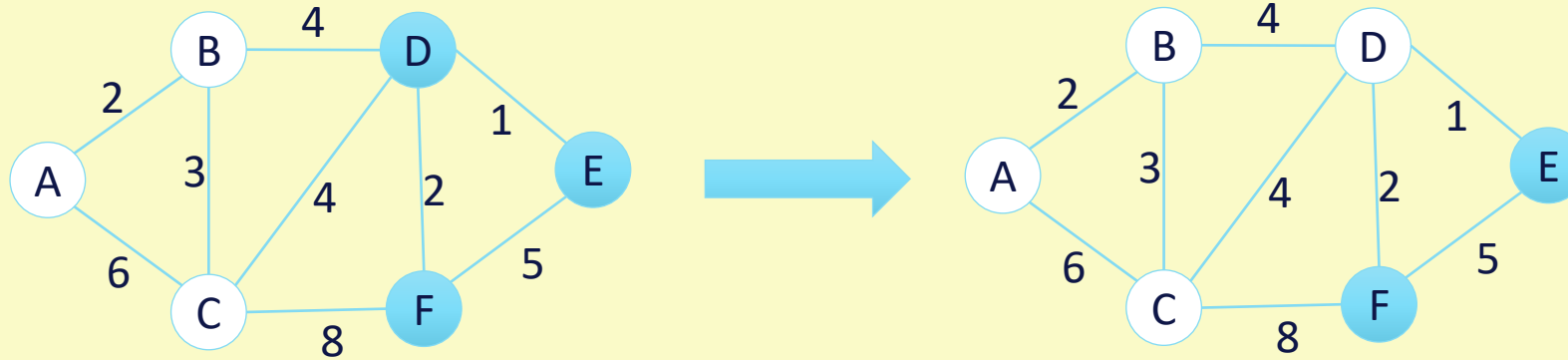
	A	B	C	D	E	F
A	-	inf	inf	inf	inf	inf
B	-	2(-)	6	inf	inf	inf
C	-	-	5(-)	6	inf	inf

$A \rightarrow B = 2$
 $A \rightarrow B \rightarrow C = 5$

$S = \{A, B, C\}$ $U = \{D, E, F\}$

$A \rightarrow B \rightarrow C = 5$
 $A \rightarrow B \rightarrow D = 6$
 $A \rightarrow B \rightarrow E = \text{inf}$
 $A \rightarrow B \rightarrow F = \text{inf}$

Dijkstra Example



Source: A, Destination: F

$S = \{A, B, C\}$ $U = \{D, E, F\}$

	A	B	C	D	E	F
A	-	inf	inf	inf	inf	inf
B	-	2(-)	6	inf	inf	inf
C	-	-	5(-)	6	inf	inf
D	-	-	-	6(-)	inf	13

$S = \{A, B, C, D\}$ $U = \{E, F\}$

($A \rightarrow B \rightarrow D = 6$) < ($A \rightarrow B \rightarrow C \rightarrow D = 9$), not update

$A \rightarrow B \rightarrow C \rightarrow E = \text{inf}$,

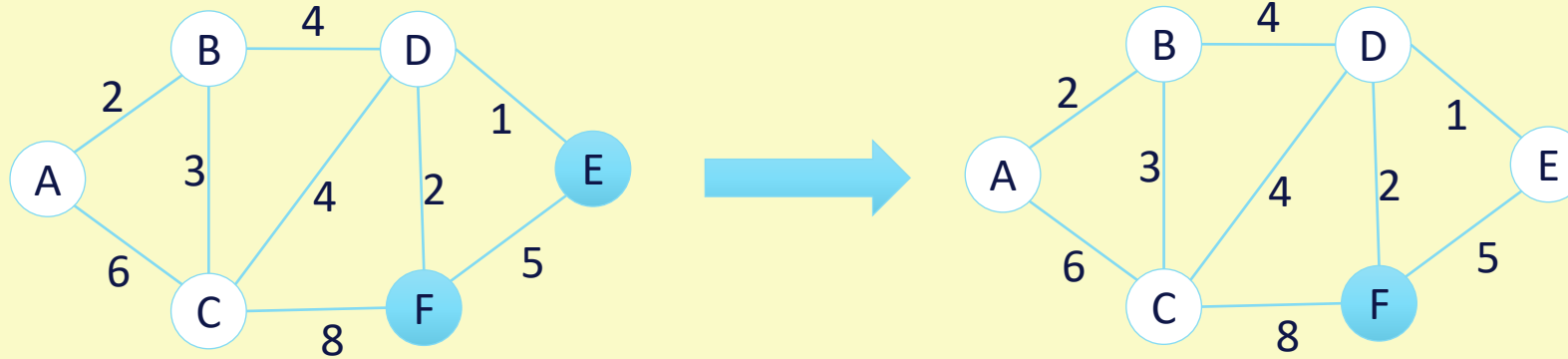
$A \rightarrow B \rightarrow C \rightarrow F = 13$

$A \rightarrow B = 2$

$A \rightarrow B \rightarrow C = 5$

$A \rightarrow B \rightarrow D = 6$

Dijkstra Example



Source: A, Destination: F

$S = \{A, B, C, D\}$ $U = \{E, F\}$

	A	B	C	D	E	F
A	-	inf	inf	inf	inf	inf
	-	2(-)	6	inf	inf	inf
	-	-	5(-)	6	inf	inf
	-	-	-	-	inf	13
	-	-	-	-	7(-)	8

$A \rightarrow B = 2$

$A \rightarrow B \rightarrow C = 5$

$A \rightarrow B \rightarrow D = 6$ $A \rightarrow B \rightarrow D \rightarrow E = 7$

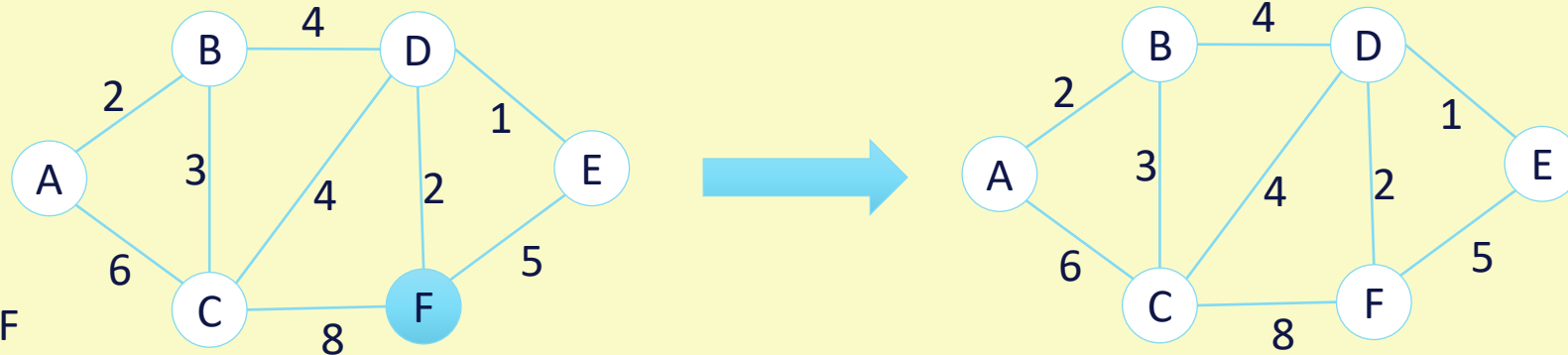
$S = \{A, B, C, D, E\}$ $U = \{F\}$

$(A \rightarrow B \rightarrow D \rightarrow E = 7) < (A \rightarrow B \rightarrow C \rightarrow E = \text{inf})$

$A \rightarrow B \rightarrow D \rightarrow F = 8$



Dijkstra Example



Source: A, Destination: F

$S = \{A, B, C, D, E\}$ $U = \{F\}$

$S = \{A, B, C, D, E, F\}$ $U = \{\}$

	A	B	C	D	E	F
A	-	inf	inf	inf	inf	inf
A	-	2(-)	6	inf	inf	inf
A	-	-	5(-)	6	inf	inf
A	-	-	-	9(-)	inf	13
A	-	-	-	-	7(-)	8
A	-	-	-	-	-	8(-)

$(A \rightarrow B \rightarrow D \rightarrow E \rightarrow F = 12) > (A \rightarrow B \rightarrow D \rightarrow F = 8)$

$A \rightarrow B = 2$

$A \rightarrow B \rightarrow C = 5$

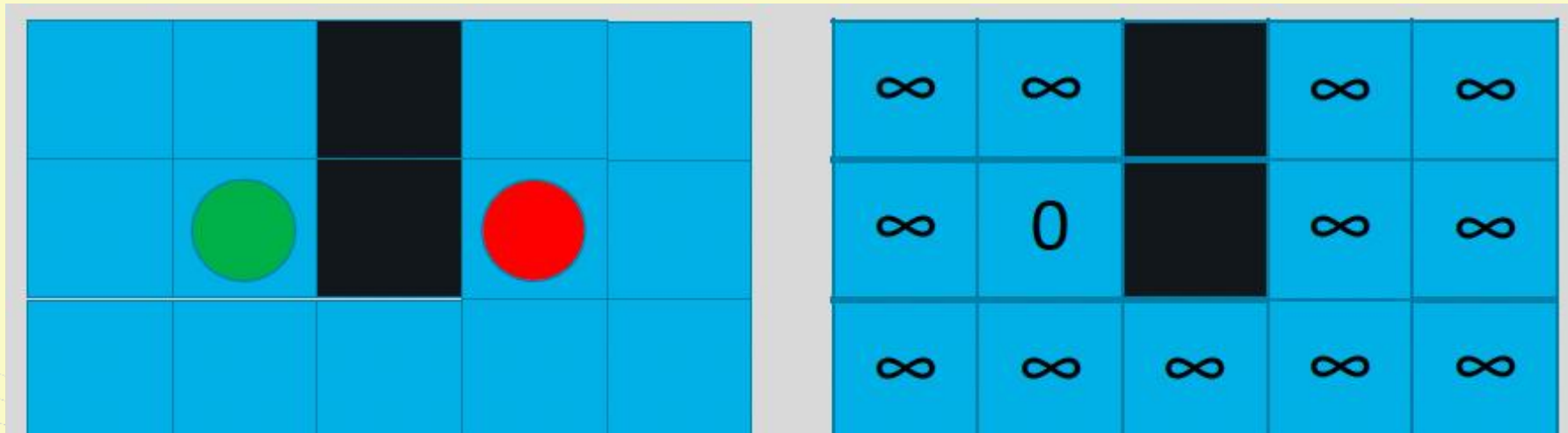
$A \rightarrow B \rightarrow D = 6$ $A \rightarrow B \rightarrow D \rightarrow E = 7$ $A \rightarrow B \rightarrow D \rightarrow F = 8$



UNIVERSITY
OF HULL

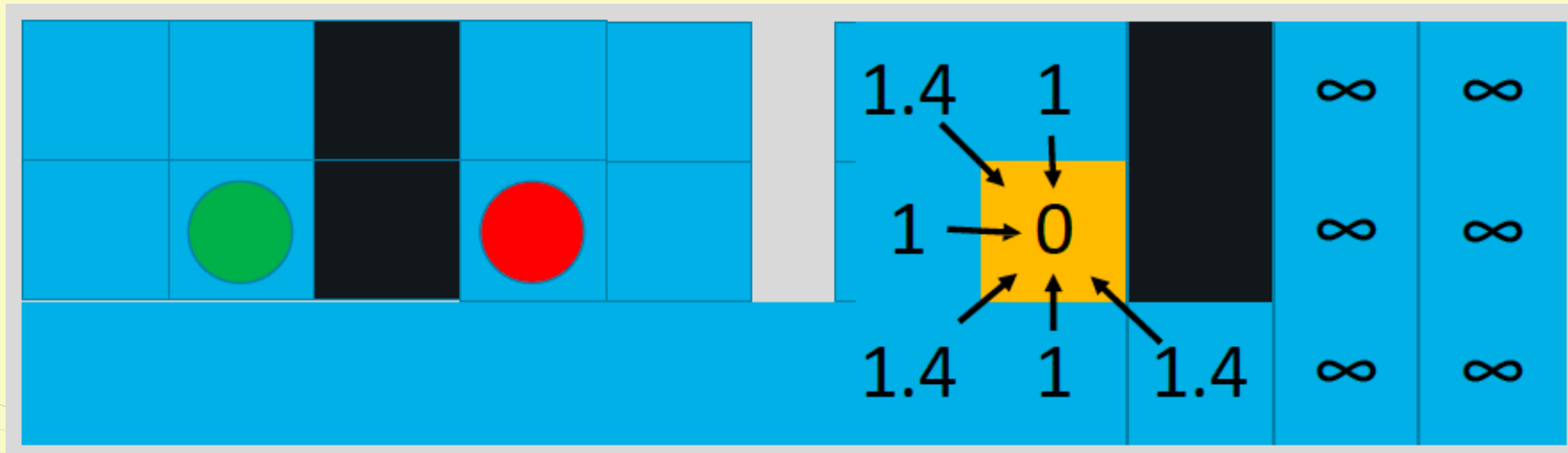
Plan a path using Dijkstra

- Green: source
- Red: destination
- Black: Obstacle
- Move direction: 8
- Vertical/horizontal weight is 1, cross weight 1.4



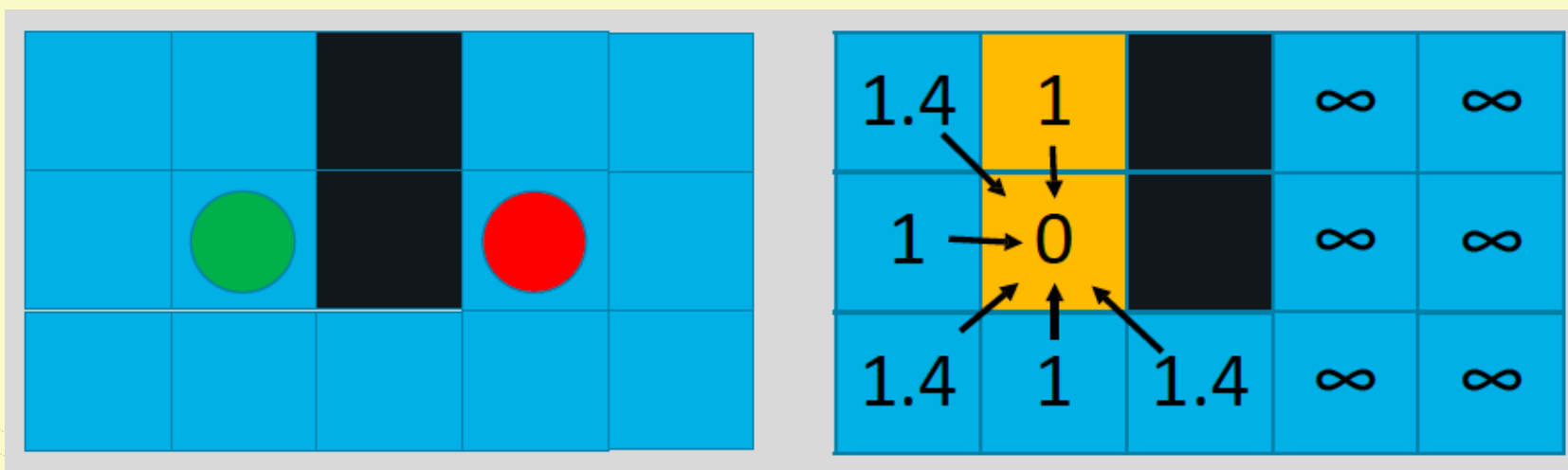
Plan a path using Dijkstra

- Update adjacent neighbors
- Orange: visited



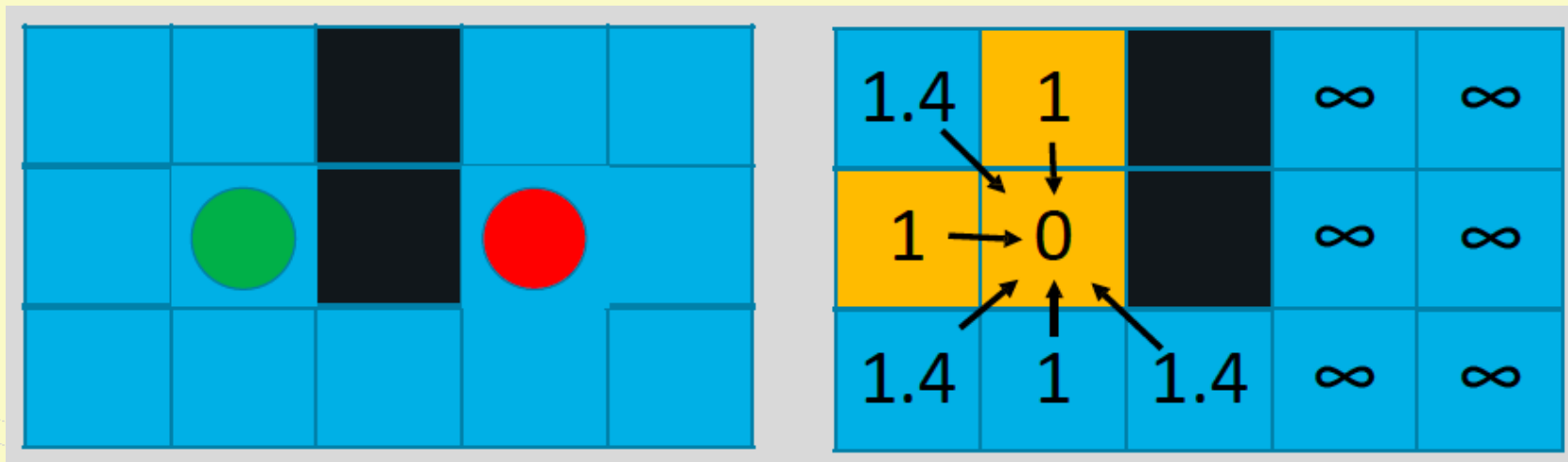
Plan a path using Dijkstra

- Choose minimum
- Recalculate neighbors



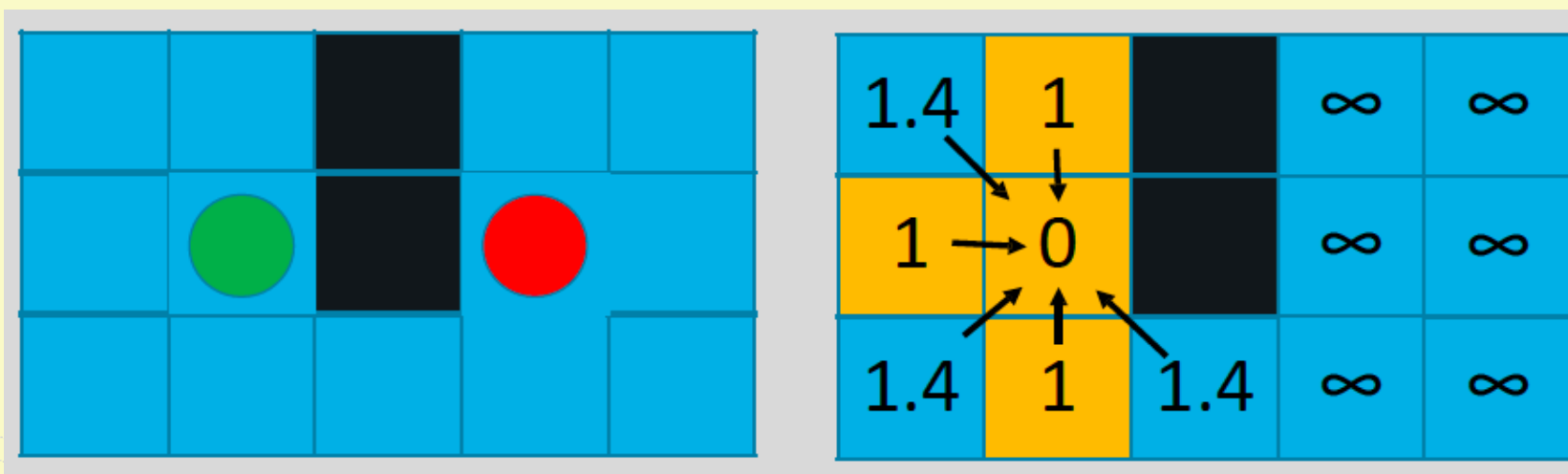
Plan a path using Dijkstra

- Choose Minimum (Now we have two left so pick randomly)
- Update neighbours



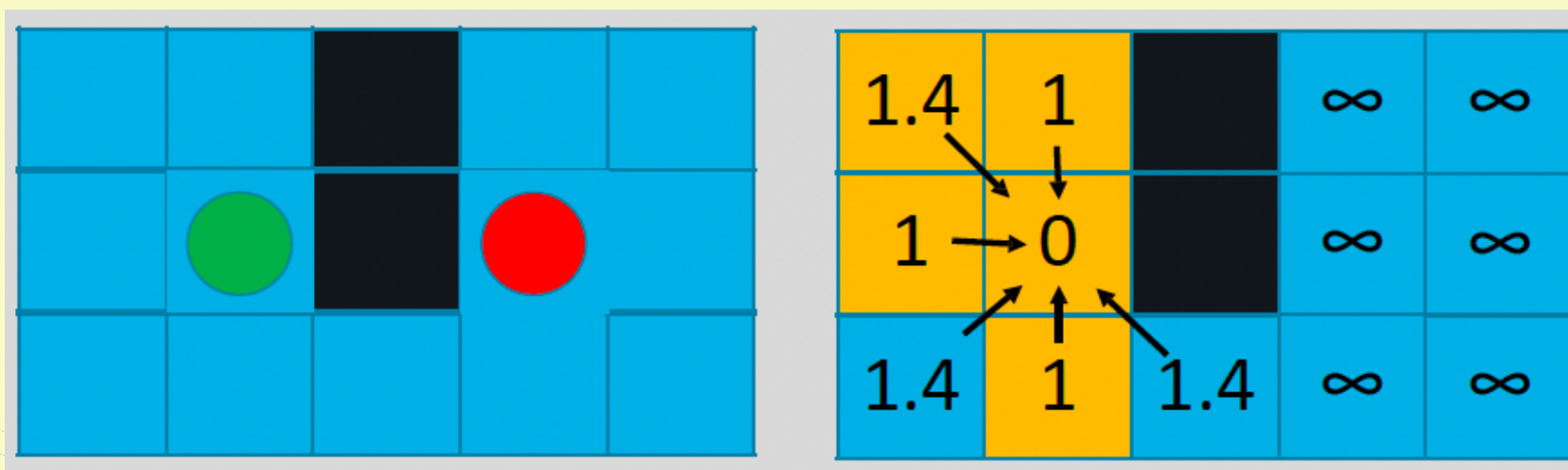
Plan a path using Dijkstra

- Choose Minimum
- Update neighbours



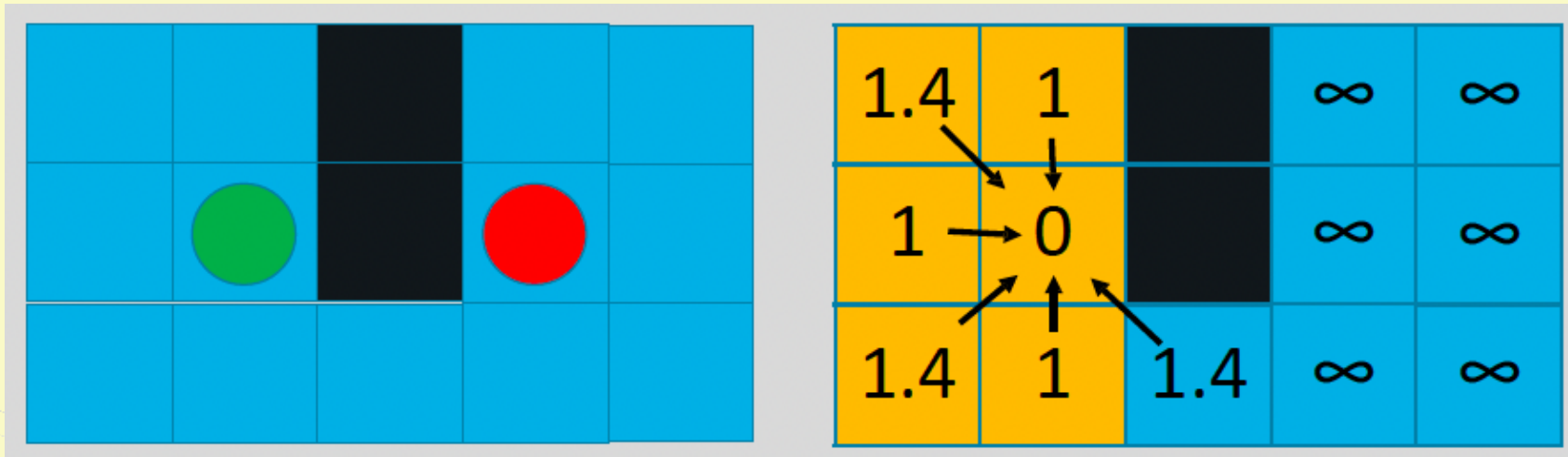
Plan a path using Dijkstra

- Choose Minimum (randomly)
- Update neighbours



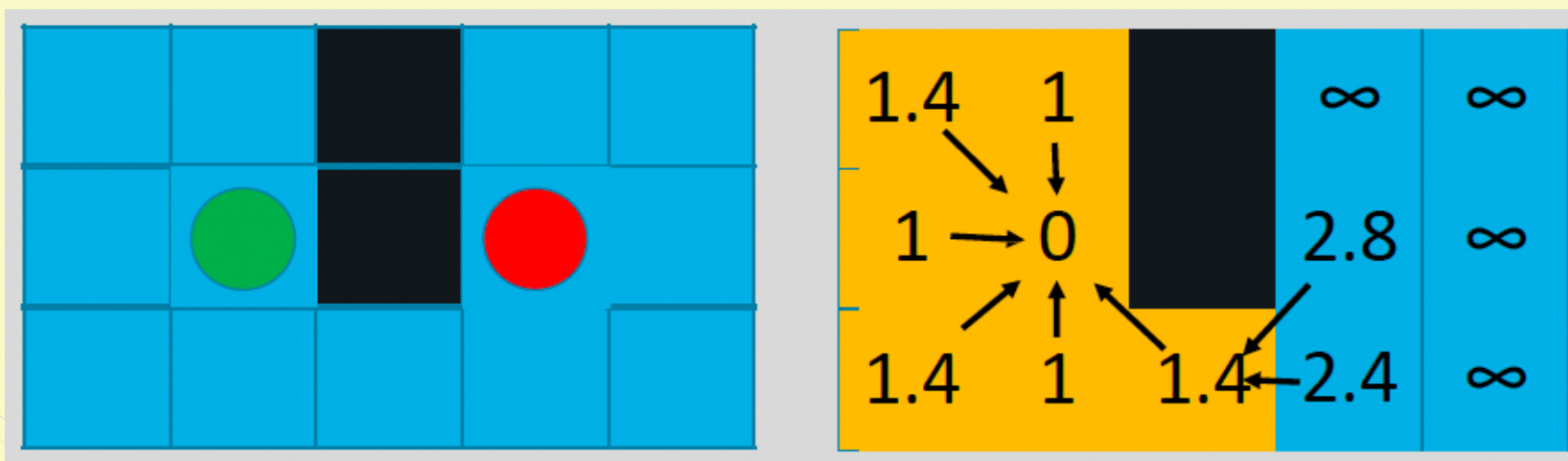
Plan a path using Dijkstra

- Choose Minimum
- Update neighbours



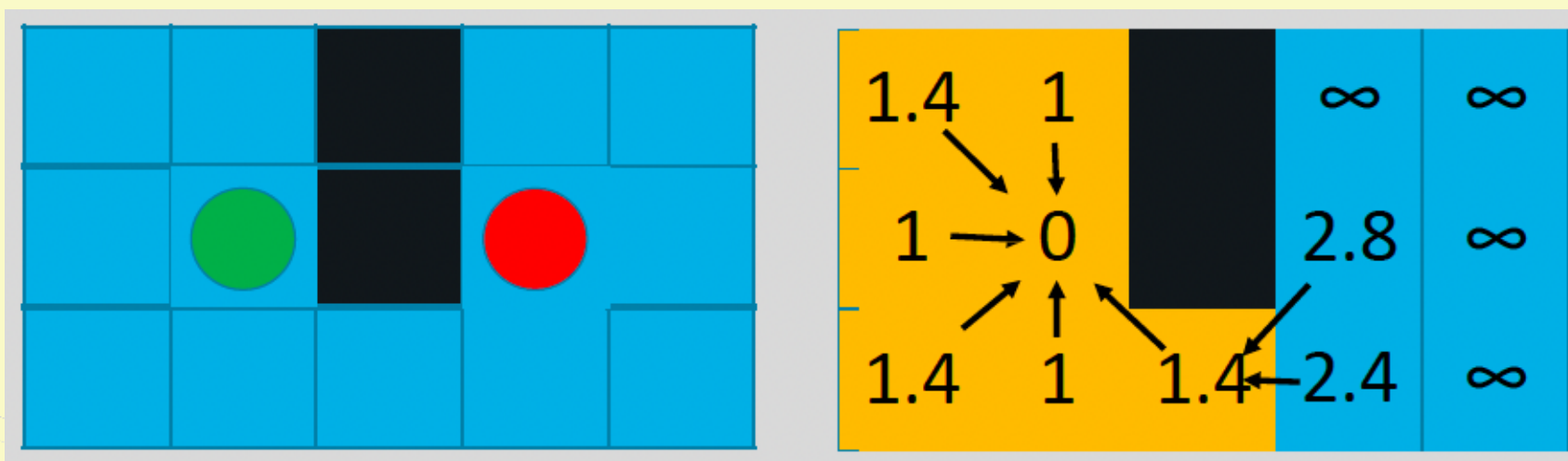
Plan a path using Dijkstra

- Choose Minimum
- Update neighbours



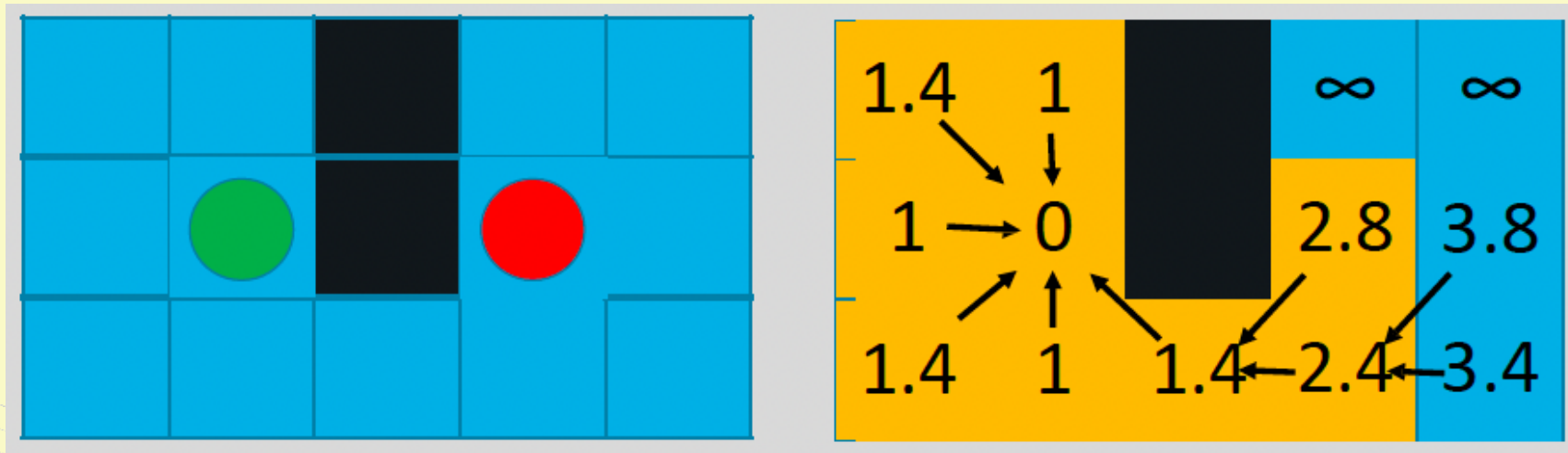
Plan a path using Dijkstra

- Choose Minimum
- Update neighbours



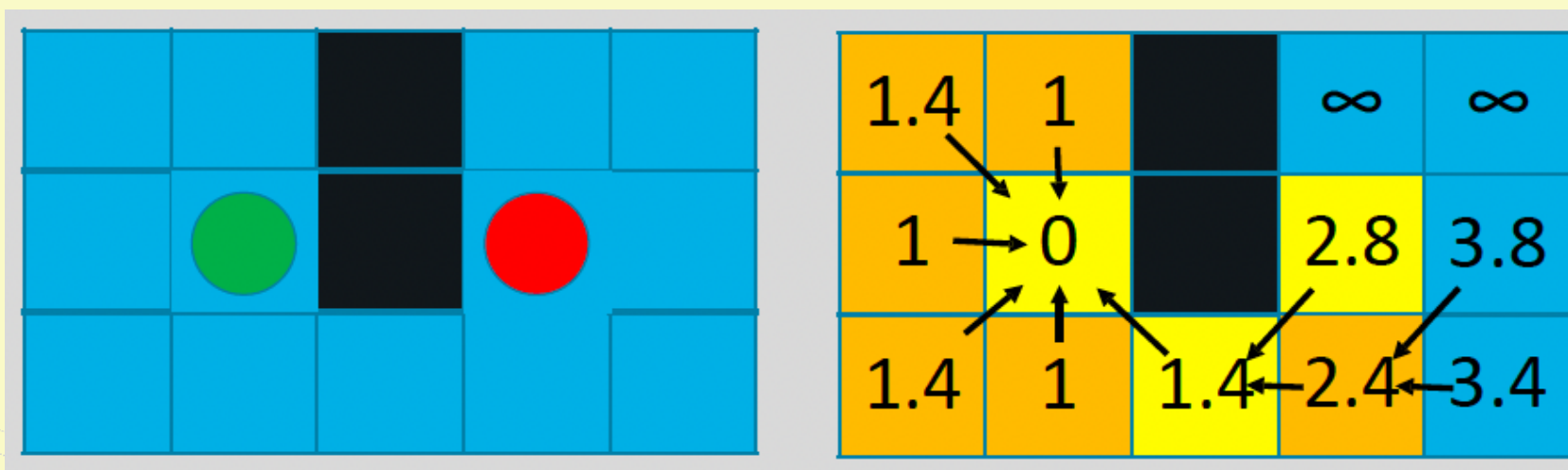
Plan a path using Dijkstra

- Choose Minimum
- Update neighbours



Plan a path using Dijkstra

- Destination reached





UNIVERSITY
OF HULL

A* Algorithm



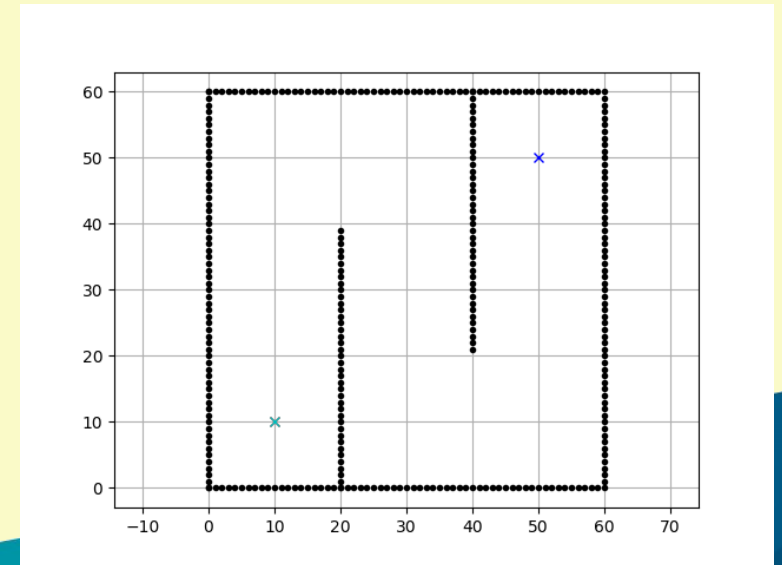
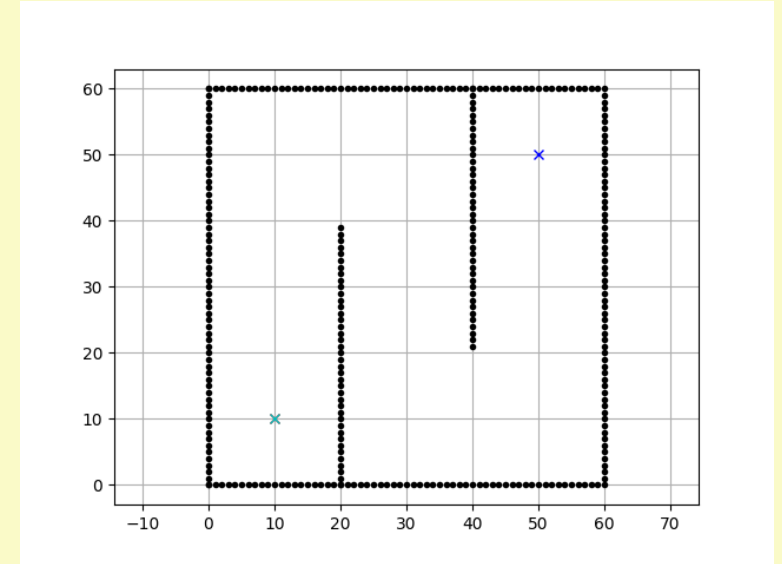
UNIVERSITY
OF HULL

A* Algorithm

- Like A*, but uses a heuristic to guess how close we are to the end
 - For spatial graphs can use Euclidian Distance
 - For grids Manhattan Distance

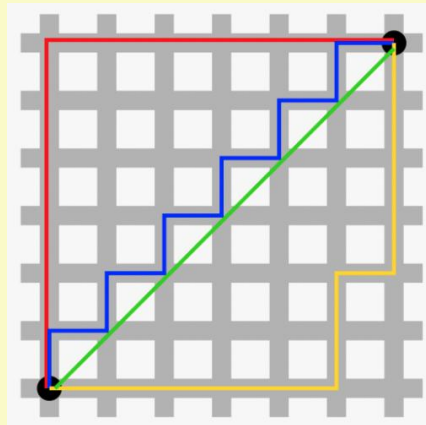
A* Algorithm

- Dijkstra doesn't consider target position
- It will explore the entire space
- A* algorithm:
 - Use a heuristic approach to **estimate** the total cost of a path
 - Estimation is not necessary correct
 - Similar to Dijkstra



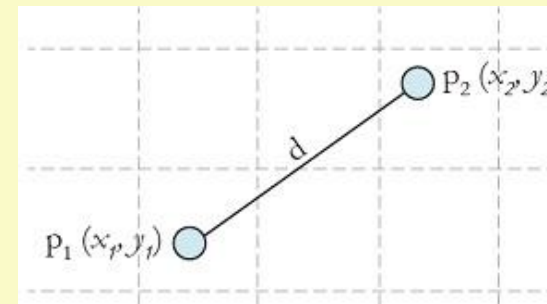
A* Heuristic

- $f(n) = g(n) + h(n)$
- Where
 - $f(n)$ = total estimated cost of path through node n
 - $g(n)$ = cost so far to reach node n
 - $h(n)$ = estimated cost from n to goal. This is the heuristic part of the cost function, so it is like a guess.
 - What if $h(n)$ is constant 0?



$$h = |x_{start} - x_{destination}| + |y_{start} - y_{destination}|$$

Manhattan distance



$$h = \sqrt{(x_{start} - x_{destination})^2 + (y_{start} - y_{destination})^2}$$

Euclidean distance



A* Algorithm pseudocode

```
make an openlist containing only the starting node
make an empty closed list
while (the destination node has not been reached):
    consider the node with the lowest f score in the open list
    if (this node is our destination node) :
        we are finished
    if not:
        put the current node in the closed list and look at all of its neighbors
        for (each neighbor of the current node):
            if (neighbor has lower g value than current and is in the closed list) :
                replace the neighbor with the new, lower, g value
                current node is now the neighbor's parent
            else if (current g value is lower and this neighbor is in the open list ) :
                replace the neighbor with the new, lower, g value
                change the neighbor's parent to our current node

        else if this neighbor is not in both lists:
            add it to the open list and set its g
```

A* Algorithm example

					D
S					

S has no parent node, so $g(s) = 0$;
 Calculate Euclidean distance $h(s)=10$;
 $f(s) = 10$;
 Only s in openlist, so min is S, move to
 close list

Openlist: S

Closetlist:

Openlist:

Closetlist:S

A* Algorithm example

					D
N1	N2				
S					

Check neighbours of S, add N1 and N2 to open list.

N1 and N2 has parent node s

$g(N1) = 1$, $h(N1)=6.4$, $f(N1) = 7.4$;

$g(N2)=1.4$, $h(N2)=5.65$, $f(N2) = 7.05$;

Minimum is N2, so move N2 to close list

Openlist: N1, N2

Closetlist: S

$g(Nmin=S)=0$

Openlist: N1

Closetlist: S, N2

$g(Nmin=N2)=1.4$

A* Algorithm example

					D
N3	N4				
N1	N2	N5			
S		N6			

Check neighbours of N2: add N3,N4,N5,N6 to open list, and add N2 as their parent node

~~$g(N1) = 2.4$, $h(N1) = 6.4$, $f(N1) = 8.8$;~~

$g(N1)_{old} = 1$, $h(N1) = 6.4$, $f(N1)_{old} = 7.4$

(No update for N1 parent)

$g(N3) = 2.8$, $h(N3) = 5.8$, $f(N3) = 8.6$;

$g(N4) = 2.4$, $h(N4) = 5$, $f(N4) = 7.4$;

$g(N5) = 2.4$, $h(N5) = 5$, $f(N5) = 7.4$;

$g(N6) = 2.8$, $h(N6) = 5.8$, $f(N6) = 8.6$;

Openlist: N1, N3,N4,N5,N6

Closetlist: S, N2

$g(N_{min=N2}) = 1.4$

Minimum is N4 or N5, random pick one N4,
so move N4 to close list

Openlist: N1, N3, N5, N6

Closetlist: S, N2, N4

$g(N_{min=N4}) = 7.4$

A* Algorithm example

					D
N7	N8	N9			
N3	N4				
N1	N2	N5			
S		N6			

Check neighbours of N4: add N7,N8,N9 to open list, and add N4 as their parent node
f(N1)_old, f(N3)_old, f(N5)_old stay unchanged

(No update for their parent)

$g(N7)=8.8$, $h(N7)=5.4$, $f(N7) = 14.2$;

$g(N8) = 8.4$, $h(N8)=4.5$, $f(N8) = 12.9$;

$g(N9)=8.8$, $h(N9)=3.6$, $f(N9) = 12.4$;

Minimum is N9, so move N9 to close list

Openlist: N1, N3, N5, N6

Closetlist: N2, N4

$g(N_{min}=N4)=7.4$

Openlist: N1, N3, N5, N6, N7, N8

Closetlist: S, N2, N4, N9

$g(N_{min}=N9)=12.4$

A* Algorithm example

					D
	N10	N11			
N7	N8	N9	N12		
N3	N4		N13		
N1	N2	N5			
S		N6			

Check neighbours of N9: add
N10,N11,N12,N13 to open list, and add N9 as
their parent node

$f(N8)_{old}$ remains unchanged

(No update for their parent)

$g(N10)=13.8$, $h(N10)=4.1$, $f(N10) = 17.9$;

$g(N11) = 13.4$, $h(N11)=3.2$, $f(N11) = 16.6$;

$g(N12)=13.4$, $h(N12)=2.8$, $f(N12) = 16.2$;

$g(N13)=13.8$, $h(N13)=3.6$, $f(N13) = 17.4$;

Openlist: N1, N3, N5, N6, N7, N8

Closetlist: S, N2, N4, N9

$g(N_{min}=N9)=12.4$

Minimum is N12, so move N12 to close list

Openlist: ... N10, N11, N13

Closetlist: S, N2, N4, N9, N12

$g(N_{min}=N12)=16.2$

A* Algorithm example

					D
	N10	N11		N14	
N7	N8	N9	N12	N15	
N3	N4		N13		
N1	N2	N5			
S		N6			

Check neighbours of N12: add N14, N15 to open list, and add N12 as their parent node
 $g(N14)=17.6$, $h(N14)=1.4$, $f(N14) = 19$;
 $g(N15) = 17.2$, $h(N15)=2.2$, $f(N15) = 19.4$;

Minimum is N14, so move N14 to close list

Openlist: ... N10, N11, N13
 Closelist: S, N2, N4, N9, N12
 $g(N_{min}=N12)=16.2$

Openlist: ... N10, N11, N13, N15
 Closelist: S, N2, N4, N9, N12, N14
 $g(N_{min}=N14)=19$

A* Algorithm example

			N16	N17	D
	N10	N11		N14	N18
N7	N8	N9	N12	N15	N19
N3	N4		N13		
N1	N2	N5			
S		N6			

Check neighbours of N14: add N16, N17, D, N18, N19 to open list, and add N14 as their parent node

$g(N16)=20.4$, $h(N16)=2$, $f(N16) = 22.4$;

$g(N17) = 20$, $h(N17)=1$, $f(N17) = 21$;

$g(D) = 20.4$, $h(D)=0$, $f(D) = 20.4$;

$g(N18) = 20$, $h(N18)=1$, $f(N18) = 21$;

$g(N19) = 20.4$, $h(N19)=2$, $f(N19) = 22.4$;

Minimum is D, we found the path successfully!

Openlist: ... N10, N11, N13, N15

Closetlist: S, N2, N4, N9, N12, N14

$g(N_{min}=N14)=19$

Openlist: ... N10, N11, N13, N15

Closetlist: S, N2, N4, N9, N12, N14, D

$g(N_{min}=D)=20.4$

Summary

- World representation is highly dependent on the game
 - Typically games use grids or sparse graphs
- Path finding algorithms are highly dependent on the world representation and the game
- A* widely used in many areas
- Openlist, closelist and heuristic functions
- A* doesn't like Dijkstra to search the entire space
- When A* has heuristic function $h(n)=0$, it becomes Dijkstra

Decision Making Strategies



UNIVERSITY
OF HULL

Finite State Machines

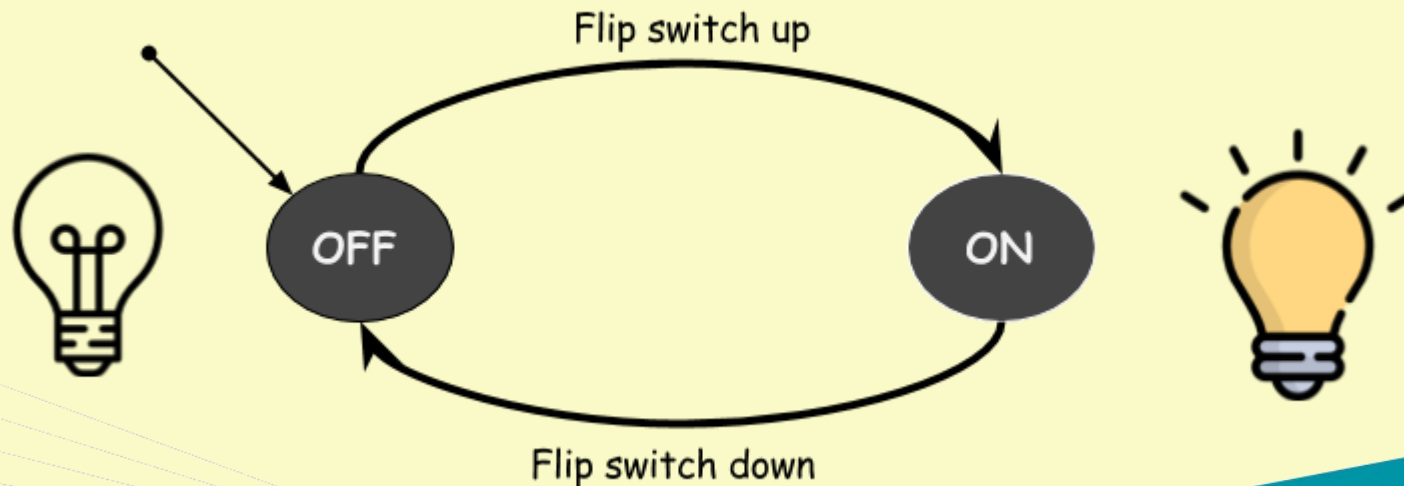
Platformer Game Example

- The player presses B to Jump
- The player shouldn't be able to jump forever though – so check to see if we're jumping already
- The player can also press down to duck, and release down to stand
- The player can jump from standing or ducking, but what happens if the jump from ducking and then release stand
- Now we want to be able to attack – but when and from where
- Using a set of conditionals is likely to lead to unmaintainable code

Finite State Machine – a definition

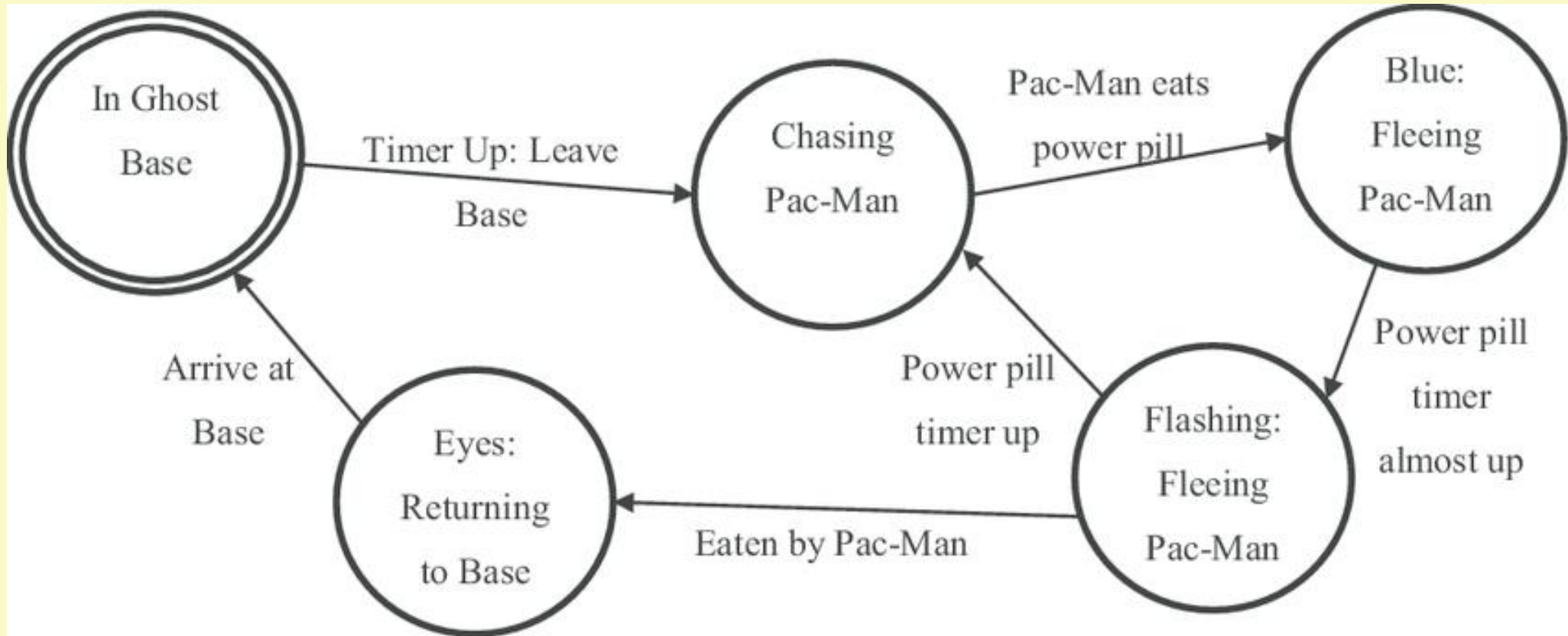
“A finite state machine is a device, or a model of a device, which has a finite number of states it can be in at any given time and can operate on input to either make transitions from one state to another or cause an output or an action to take place” (Buckland)

“A finite state machine can only be in one state at any one time” (Buckland)





State Diagrams



There is a minor mistake here – what happens if Pac-Man eats a power pill just before a Ghost leaves the base?

State Transition Tables

• Current State	• Condition	• State Transition
• In Ghost Base	• Timer Up: Leave Base	• Chasing Pac-Man
• Chasing Pac-Man	• Pac-Man eats power pill	• Blue: Fleeing Pac-Man
• Blue: Fleeing Pac-Man	• Power pill timer almost up	• Flashing: Fleeing Pac-Man
• Flashing: Fleeing Pac-Man	• Power pill timer up	• Chasing Pac-Man
• Flashing: Fleeing Pac-Man	• Eaten by Pac-Man	• Eyes: Returning to base
• Eyes: Returning to base	• Arrive at Base	• In Ghost Base

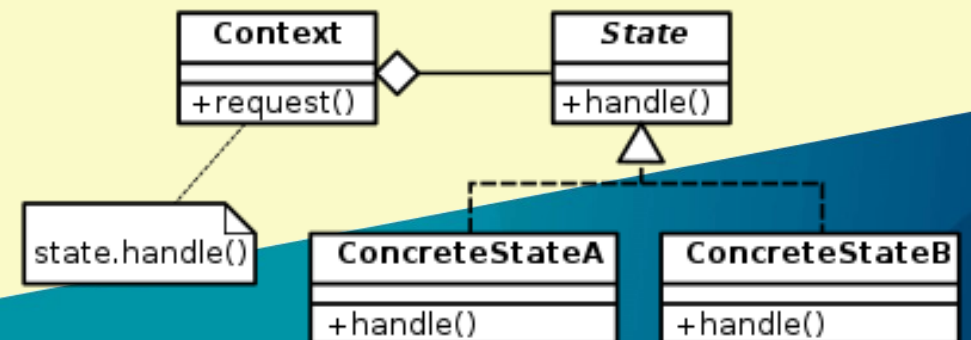


UNIVERSITY
OF HULL

The State Pattern

The State Pattern

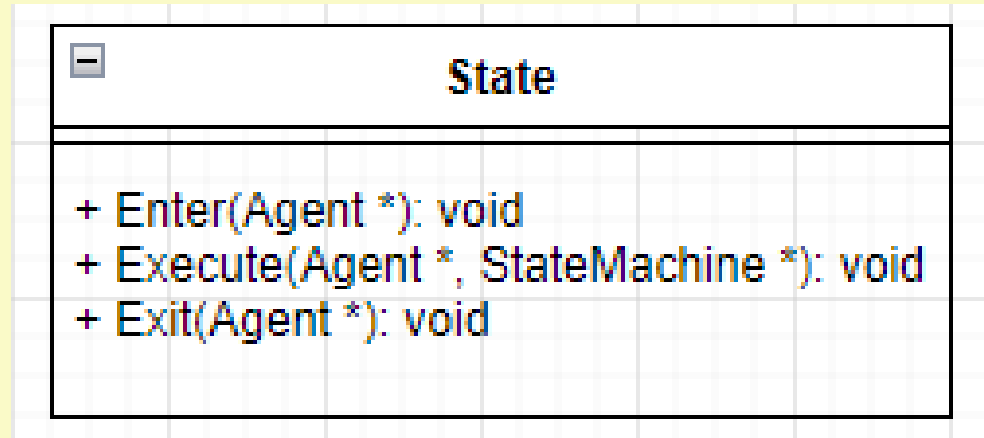
- An object should change its behaviour when its internal state changes
- State-specific behaviour should be defined independently. That is, adding new states should not affect the behaviour of existing states.
- Define separate (state) objects that encapsulate state-specific behavior for each state. That is, define an interface (state) for performing state-specific behavior, and define classes that implement the interface for each state.
- A class delegates state-specific behavior to its current state object instead of implementing state-specific behavior directly.





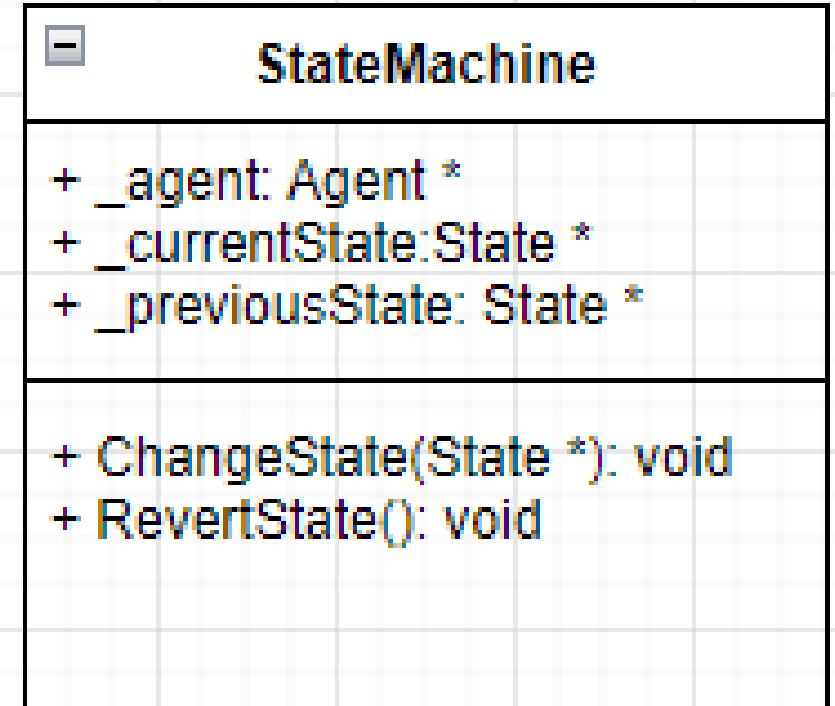
State Design – 3 E's

- Enter Method
 - Do things once when the state first begins
 - Set up variables
 - Send messages
- Execute Method
 - Checks conditions to change state
 - Sends message to change state to state machine
- Exit Method
 - Do things once when the state exits
 - Clean things up
 - Send messages



State Machine Design

- Stores current state and previous state
- ChangeState Method
 - Calls Exit on current state
 - Changes previous state to current state
 - Changes current state to new state
 - Calls Enter on new state
- ChangeToPreviousState
 - Calls ChangeState passing in previous state





UNIVERSITY
OF HULL

Hierarchical Finite State Machines

Hierarchical Finite State Machines

- As state machines get bigger they often become unwieldy
 - Suppose that if a character gets hit at any point they go into a stunned state
 - Now every state has to deal with the transition from standing, jump, falling to stunned
- HFSM groups sets of states together to simplify this process and ease the transition between groups of states
- If you use the State pattern you can use an inheritance hierarchy to place substates inside superstates
- <https://www.youtube.com/watch?v=JyF0oyarz4U>

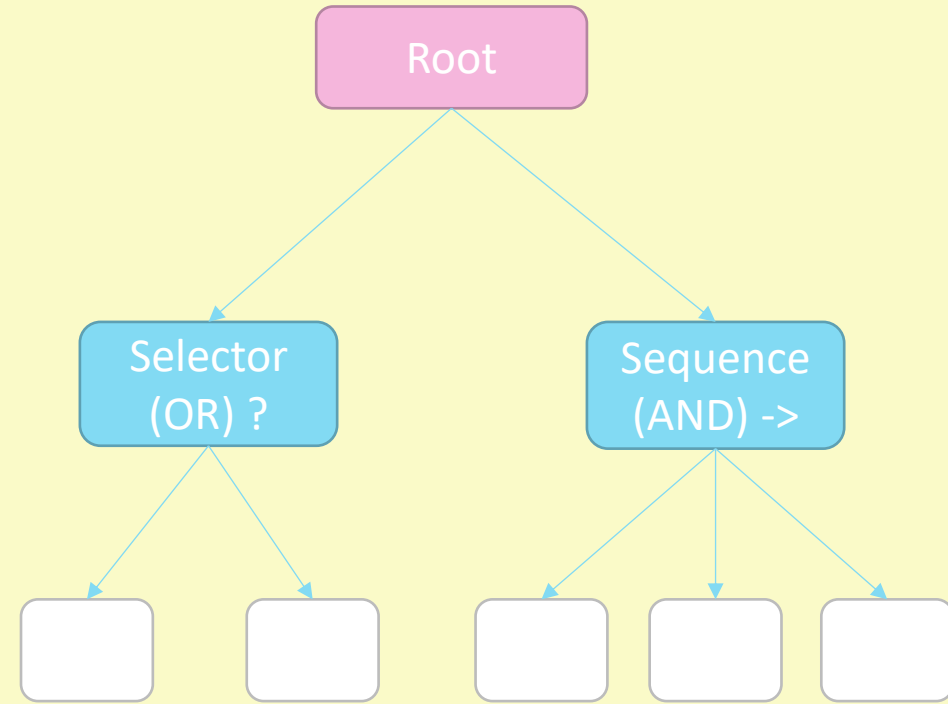


UNIVERSITY
OF HULL

Behaviour Trees

Behaviours Trees

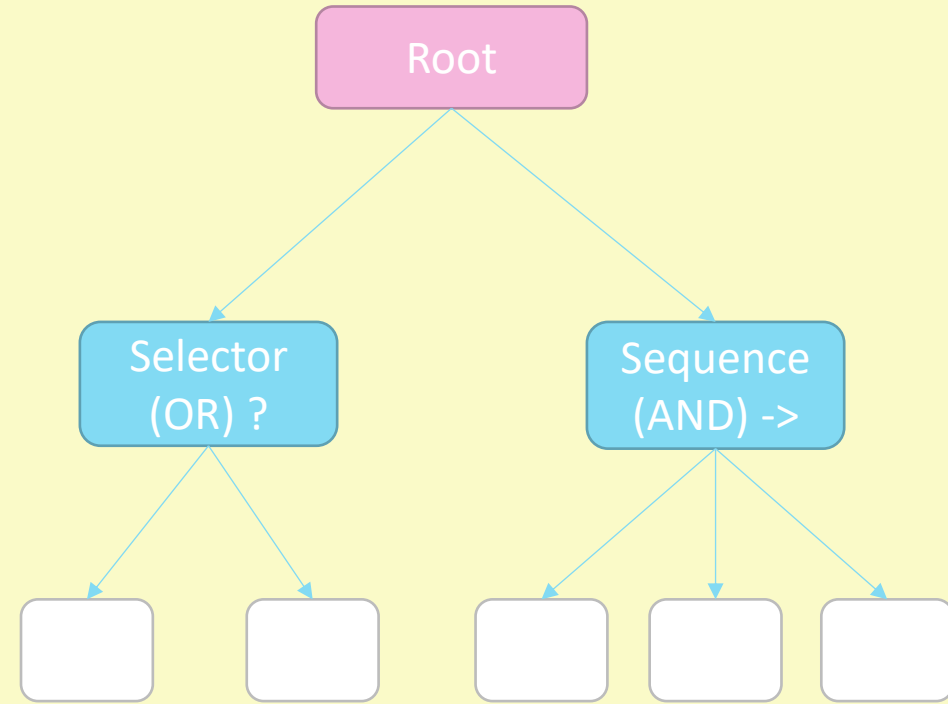
- Root
- Composite Nodes
- Leaf





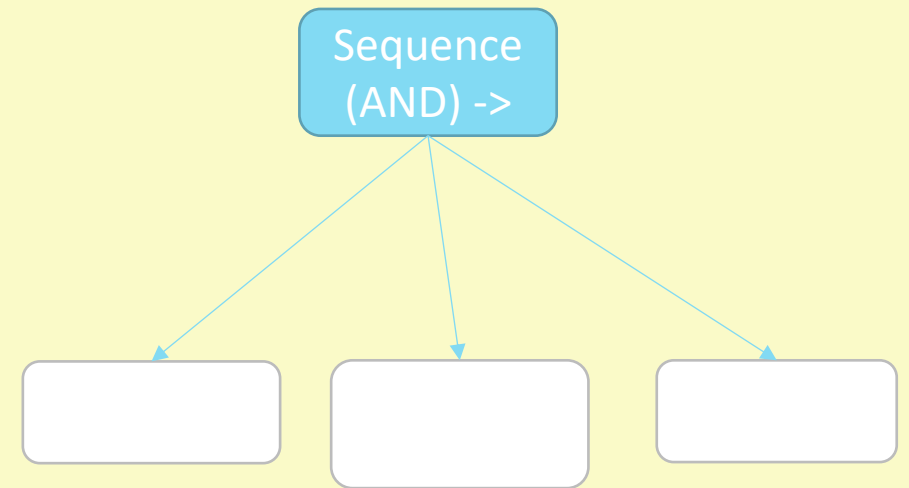
Leaf Nodes

- Executes a small amount of behaviour
 - Go to point
 - Wait 5 seconds
 - Start animation
 - Play sound
 - Fire weapon
- Can return Success, Failure or Running



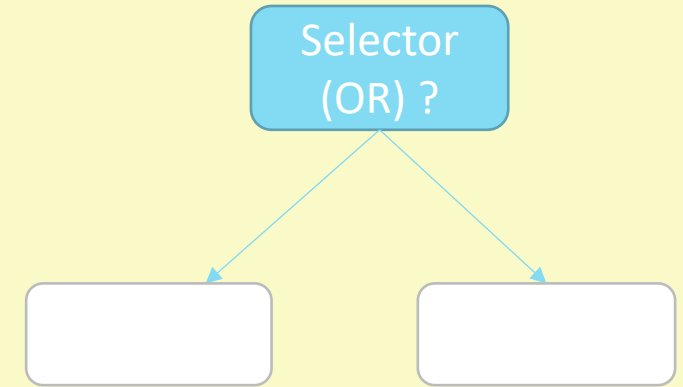
Composite Nodes - Sequence

- Sequence Nodes process children from left to right



Composite Nodes - Selector

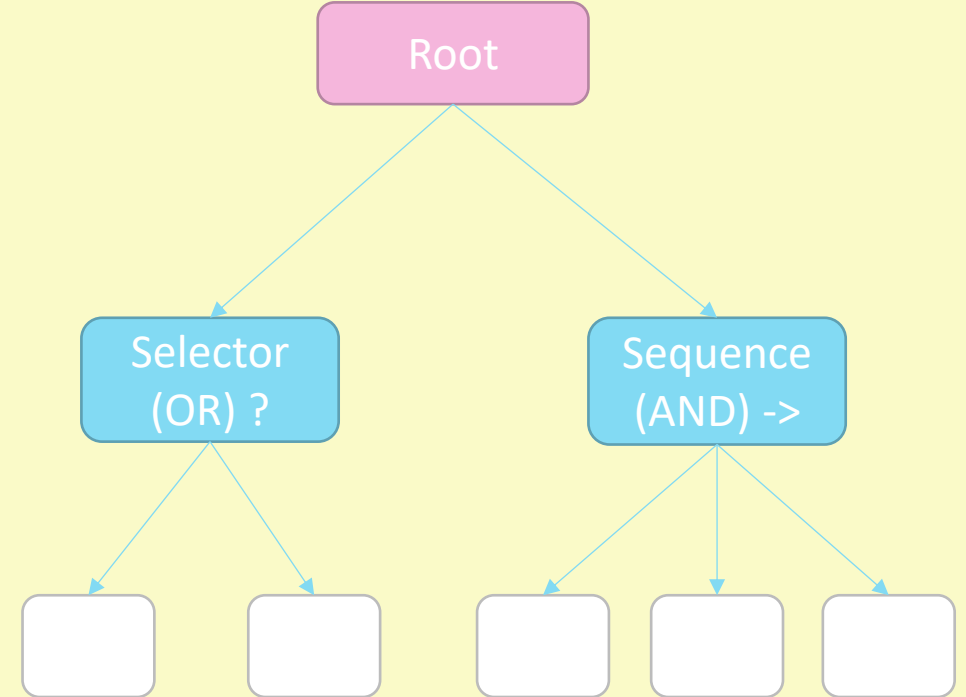
- Selector Nodes attempt to process children from left to right until one completes successfully
- For example if Fire Gun fails, perhaps because the enemy has no ammo then the agent might choose a melee attack instead





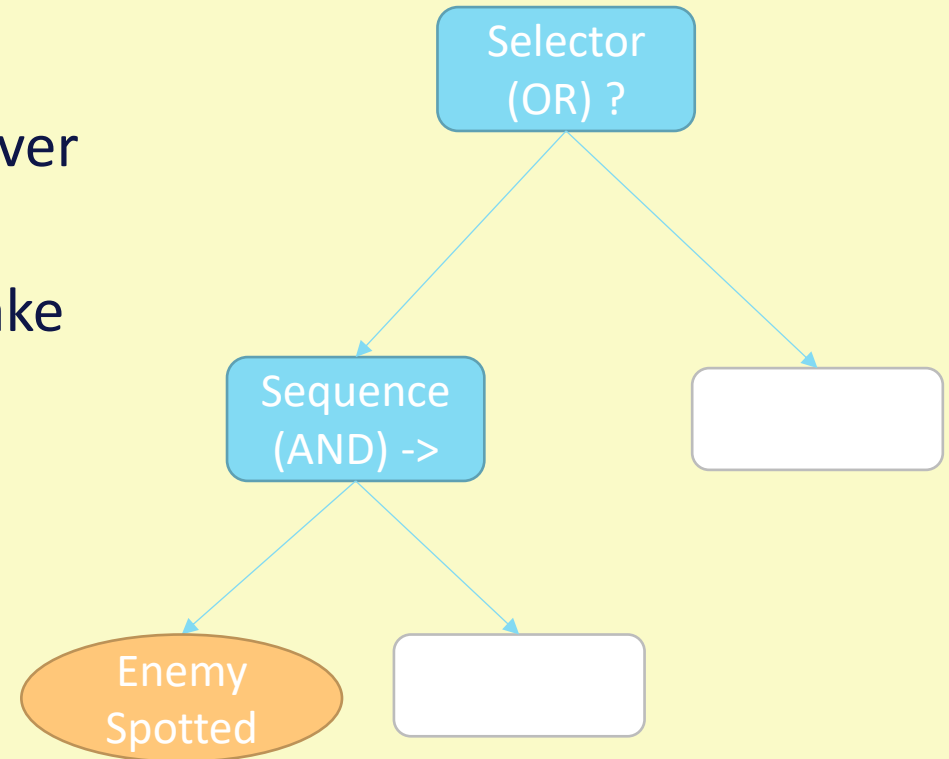
Root

- Periodically processes a tick
 - Can take a long time for large trees
- Often has reference to active node to avoid traversing the whole tree
 - Only process the current node until it returns success or failure
 - Can cause issues with reactivity - sometimes we have to force tick to update



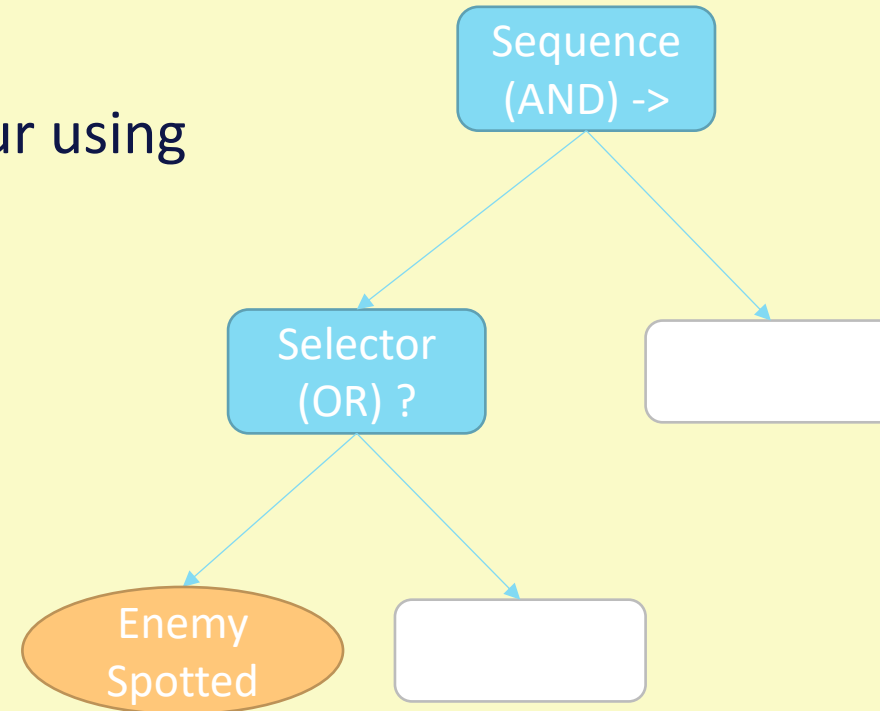
Condition Nodes

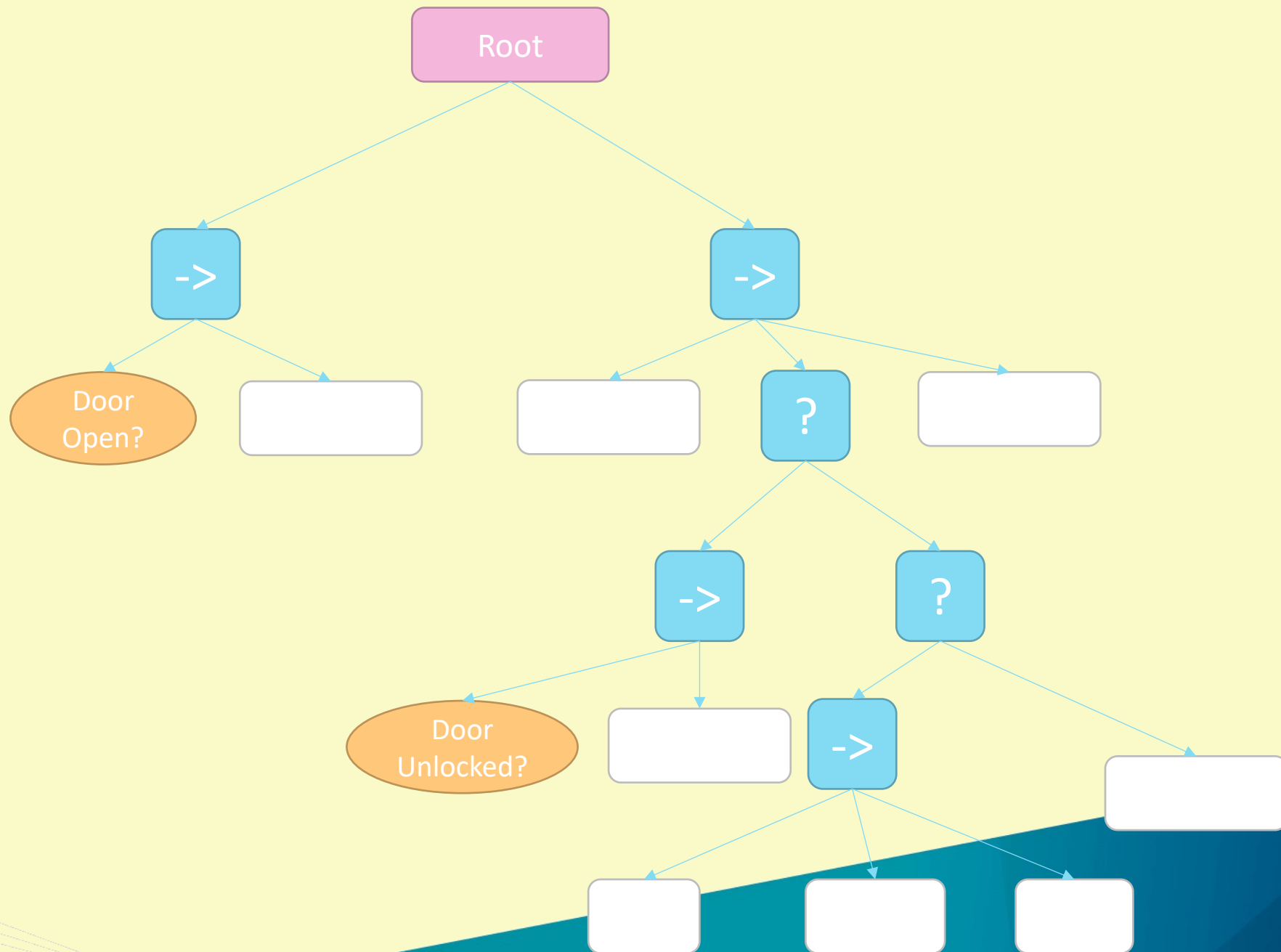
- Priority is top to bottom, left to right
- Condition nodes are a special type of node that never return running
- They can be used along with a selector node to make decisions in behaviour trees using Sequence Node



Condition Nodes

- Priority is top to bottom, left to right
- Can also be used to create fallback behaviour using Selector Node





Other Common Nodes

- Inverse
 - Turns Success to Failure and Failure to Success
- Succeed
 - Always returns success
 - Can be used to check other courses of action in a selector ?
 - Try A, Try B, Try C – if they all fail succeed anyway then move on
- Parallel
 - Execute all child nodes at the same time
- Random
 - Select a child at random



Pros and Cons

- Big Benefit
 - Modular
 - Can add, remove or replace nodes dynamically
 - Easy for designers to build
 - Fairly easy to understand
- Disadvantage
 - Inherently acyclic
 - Often end up with state dependent checks in nodes which creates dependencies

Additional Resources

- Nice introduction to behaviour trees in games
 - <https://www.youtube.com/watch?v=6VBCXvfNICM>
- Clear step by step description of behaviour trees
 - <https://www.gamedeveloper.com/programming/behavior-trees-for-ai-how-they-work>
- Behaviour tree starter kit
- Site to help understand tree builder pattern
- A variety of more complex concerns to think about
 - https://www.youtube.com/watch?v=Qq_xX1JCrel&t=2898s



UNIVERSITY
OF HULL

Goal Oriented Action Planning



- Goal Oriented Behaviour
- Agents have a list of goals – each with a priority (or *insistence*)
 - Patrol Route
 - Attack Player
 - Stay Alive/Heal
- List of available actions based on the agent and world state – each action has an associated decrease in insistence
 - Move to Position
 - Hide Behind
 - Shoot at Player

- Discontentment is a measure generated by the state after an action has successfully taken place
 - Also known as “energy metric” in Academic AI
- Discontentment focuses of a set of goals rather than any single goal (as with *insistence*)
- Lots of ways to do this
 - Sum insistence after
 - Sum square insistence after lends more weight to goals with higher insistence

GOB -> GOAP

- GOB assumes that an agent is able to perform all actions available to it
- GOAP allows *planning* of *sequences* of behaviour



UNIVERSITY
OF HULL

STRIPS

- GOAP is based on a very old technology called STRIPS
- Stanford Research Institute Problem Solver developed in 1971
- STRIPS instance includes
 - An initial state
 - Specification for goal states
 - A set of Actions
 - Each action includes preconditions and postconditions



UNIVERSITY
OF HULL

STRIPS Example

Initial state: At(A), Level(low), BoxAt(C), BananasAt(B)
Goal state: Have(bananas)

Actions:

```
// move from X to Y
_Move(X, Y)_
Preconditions: At(X), Level(low)
Postconditions: not At(X), At(Y)

// climb up on the box
_ClimbUp(Location)_
Preconditions: At(Location), BoxAt(Location), Level(low)
Postconditions: Level(high), not Level(low)

// climb down from the box
_ClimbDown(Location)_
Preconditions: At(Location), BoxAt(Location), Level(high)
Postconditions: Level(low), not Level(high)

// move monkey and box from X to Y
_MoveBox(X, Y)_
Preconditions: At(X), BoxAt(X), Level(low)
Postconditions: BoxAt(Y), not BoxAt(X), At(Y), not At(X)

// take the bananas
_TakeBananas(Location)_
Preconditions: At(Location), BananasAt(Location), Level(high)
Postconditions: Have(bananas)
```




UNIVERSITY
OF HULL

F.E.A.R

- Initially introduced in Monolith's F.E.A.R. (First Encounter Assault Recon)
- Released in 2005 by Monolith Productions
- Around 70 Actions – just 3 states
- AI in games is often to do with picking the right animation
- Often the Smart Object tells the agent how to use it

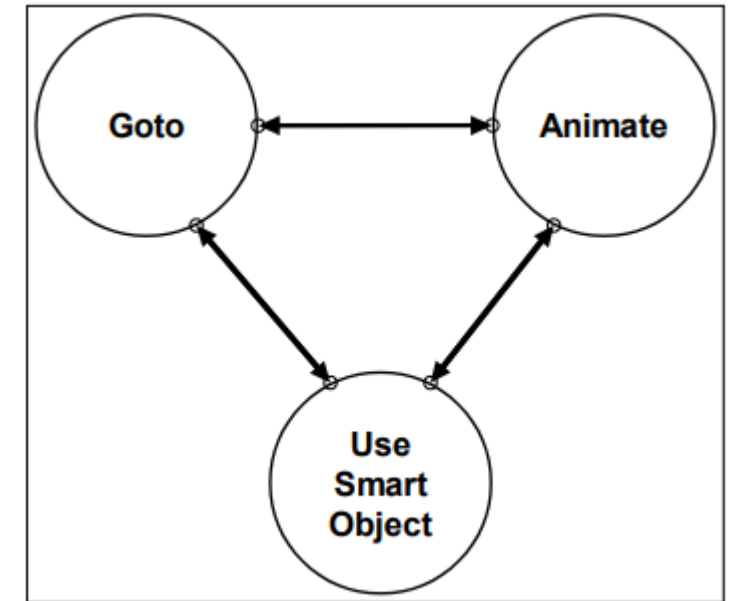


Figure 2: F.E.A.R.'s Finite State Machine

Building a plan

- Actions
 - Thing that an agent can do
 - Actions have a cost heuristic which helps select between available actions
- Preconditions
 - State required in order to begin an action
 - Need a good way to represent the world
- Effects
 - The new world state - the result of the action on the world



UNIVERSITY
OF HULL

GOAP

- Dynamically creates a graph with world states as nodes and actions as edges
- Goals are leaf nodes in the graph
- Performs Depth First Search (to a specified depth), A* or Dijkstra if there is no appropriate heuristic to find a path through the graph
 - DFS is usually sufficient and paths are typically quite short
- Goals and goal priority can be updated according to the world state

Goal Oriented Action Planning

- Can simplify finite state machines
 - FEAR had just 3 states – MoveTo, Animate and UseSpecialObject
 - Often the smart object told the Agent how it should interact
- Effectively builds a finite state machine dynamically by finding a path through potential world states using A*
 - If it is not possible to conceive a heuristic for path planning then have null heuristic and revert to Dijkstra

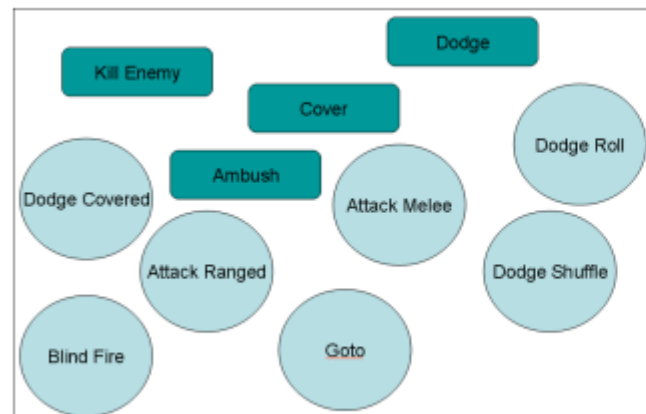
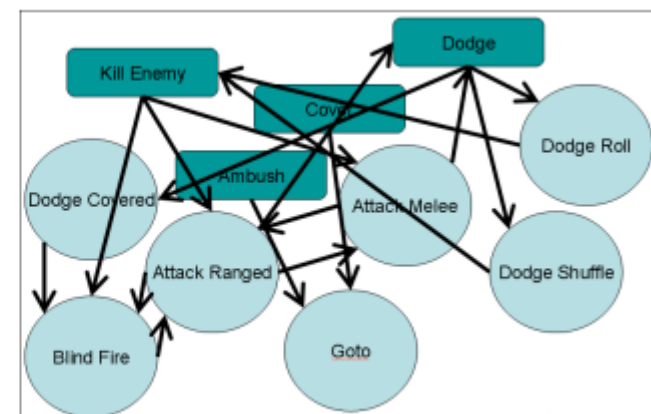


Figure 6: We do this.



But we never have to do this.



Summary

- Decision making strategies all have pros and cons
 - FSM
 - Can easily become hard to manage
 - One approach is Hierarchical FSM
 - Behaviour Trees
 - Good at sequences of behaviour
 - Difficult to get cyclic behaviour
 - GOAP
 - Effectively building an FSM at run time

Additional Resources

- [Building the AI of F.E.A.R. with Goal Oriented Action Planning | AI 101 – YouTube](#) – nice introduction to GOAP
- [Goal-Oriented Action Planning: Ten Years of AI Programming – YouTube](#) – gives some perspective on the compromises to be made in real code
- [gdc2006 orkin jeff fear \(mit.edu\)](#) – summary of GOAP in FEAR
- More on Canvas



UNIVERSITY
OF HULL

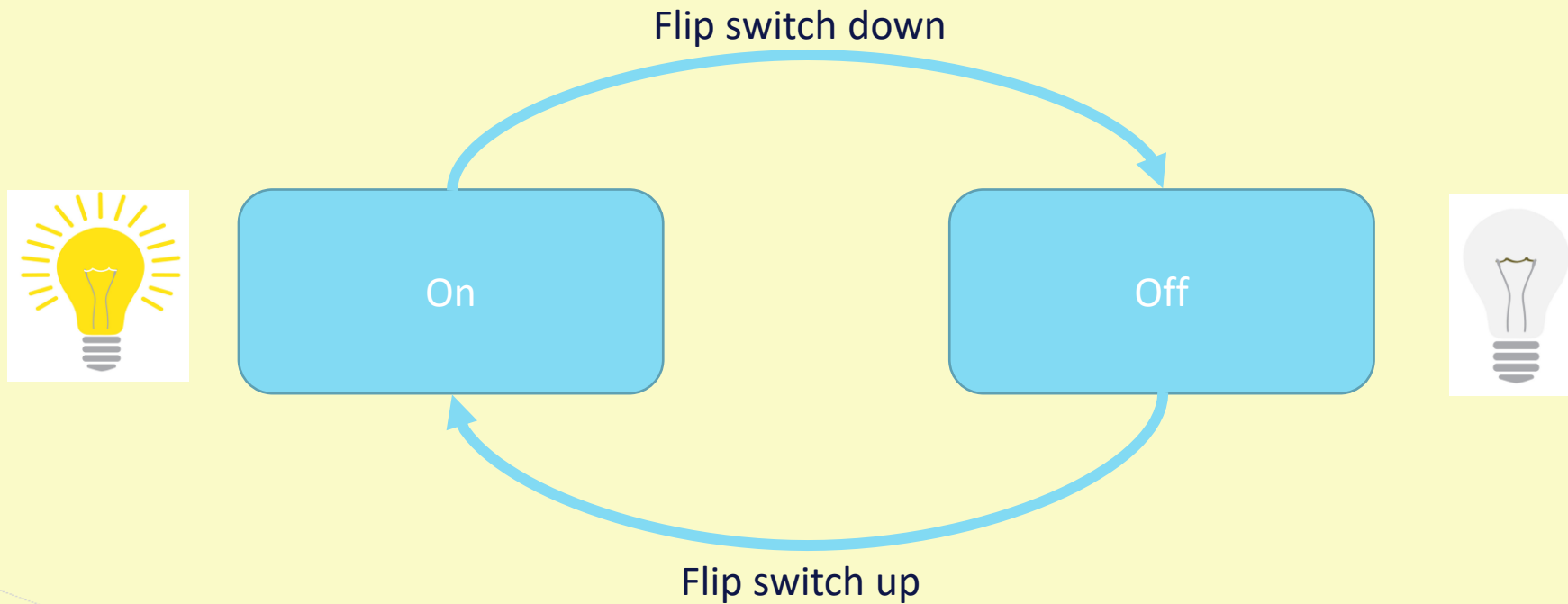
Fuzzy Logic

Crisp Logic

- Most programmes and electronic devices use ‘crisp logic’
- With crisp logic if an element of data falls within a certain range then the element is included in a set. If it is outside the range it falls in another set.
 - A character is **injured**
 - A character is **tired**
 - A character is **hungry**
- Terminology – injured, tired and hungry are examples of ***predicates***
- A character can be members of multiple sets, but sometimes ***predicates*** are mutually exclusive
 - E.g. hurt and healthy – tall and short

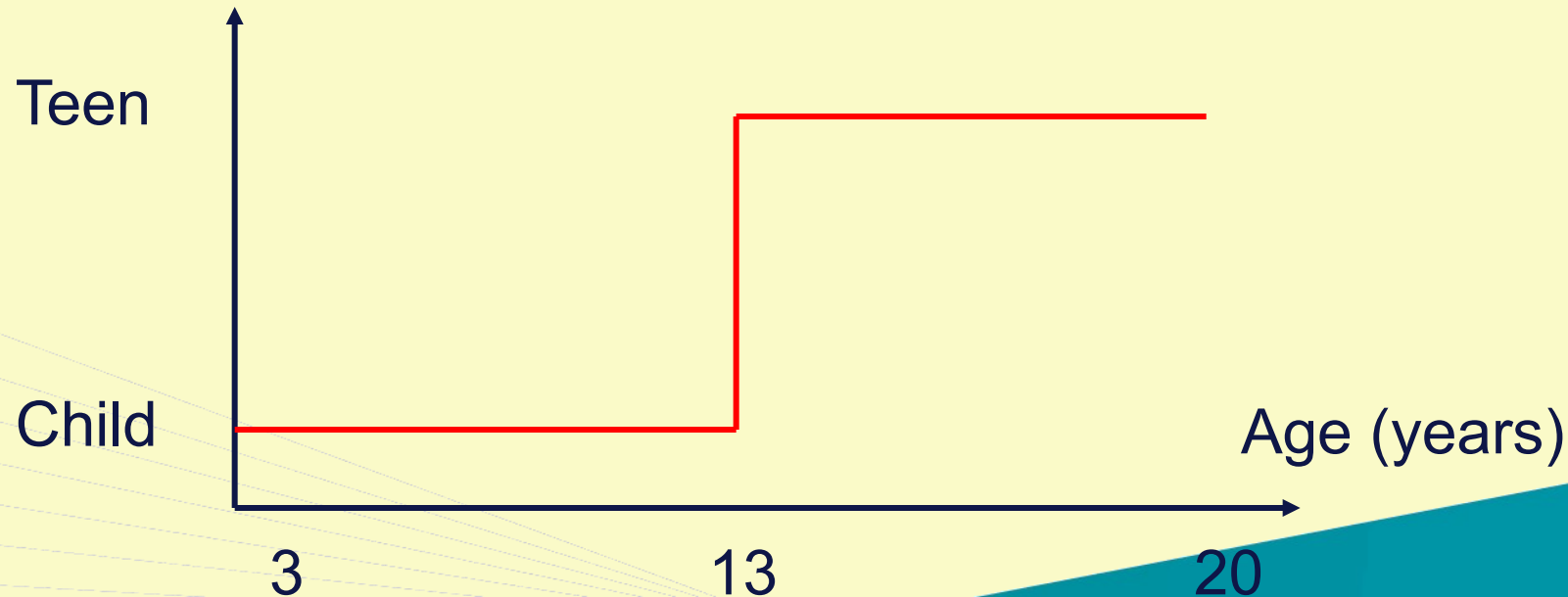
Remember our finite state machine

- In finite state machines there are a ***finite*** number of states
 - A switch is either on or it is off



Crisp Logic Example

- As an example of crisp set theory, consider a group of Youngsters aged between 3 and 19 years.
- If we want to organise them into sets of Teenagers and Children we have a crisp set solution.
- We can simply ask the subject their age if their age is greater than 12 then they are a teenager. If it is less than 13 they are in the child set.



Crisp Logic Maths

- Mathematically this problem is defined as follows:
- Suppose there are 11 in the group to be classified, with the following ages.
- $\{ 3,4,7,10,11,12,13,14,15,16,18 \}$
- Then the child set $A = \{3,4,7,10,11,12\}$
- The teen set $B = \{13,14,15,16,18\}$

Membership

- Fuzzy logic or fuzzy set theory is a method of analysing data such that elements of data can have partial inclusion
- Introduces the idea that something can have **membership** of a group
 - Typically represented with a number from 0.0f-1.0f – although sometimes 0-255
- In Fuzzy Logic the objects are not contained in a defined set, they are described by the degree of membership (DOM) of a certain class

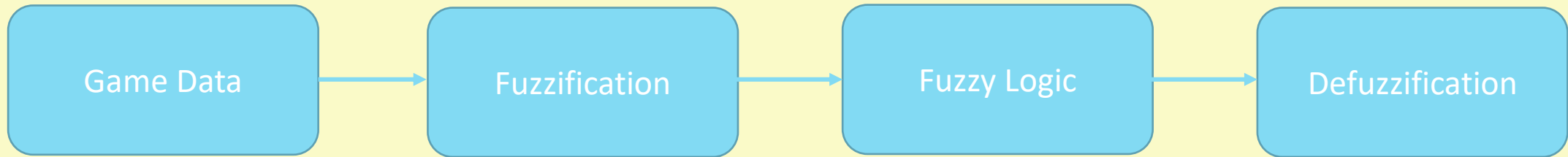


Membership

- As an example consider a group of people that we want to place in sets of Young and Old
- Using crisp logic we would have to pick an age, (50) and make all people below 50 young and all above old

Degree of Membership

- Using fuzzy logic we can define DOM of each set
- So a 70 year old would have a DOM of 1 for the old set
- A 12 year old would have a DOM of 1 for the young set
- Or a DOM of 0 for the old set
- A 50 year old would have a DOM of 0.5 for the old set
- This concept is best described by a graph of DOM against age



- Crisp game data
- Fuzzified into a series of sets of degrees of membership
- Perform fuzzy logic operations

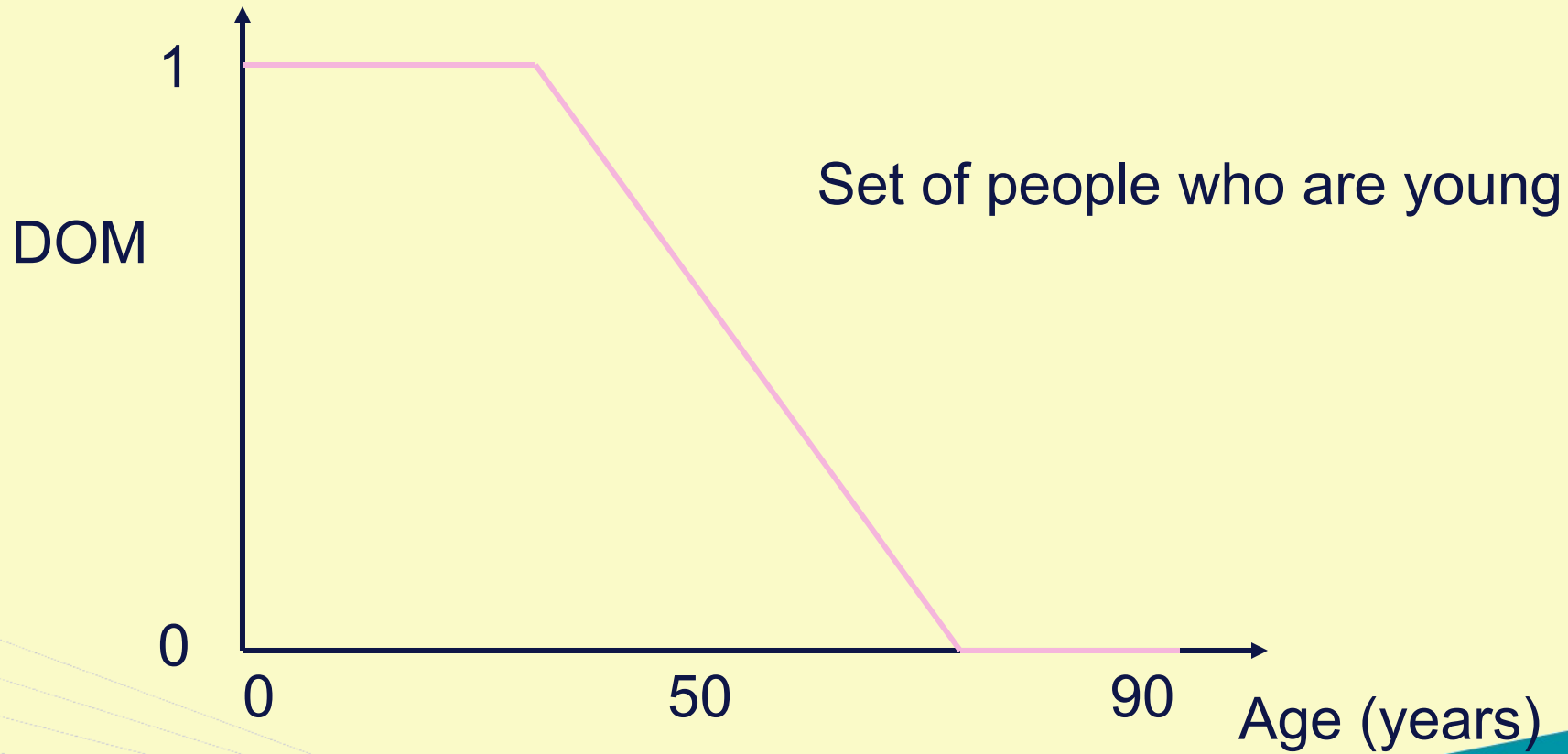


UNIVERSITY
OF HULL

Fuzzy Logic



Fuzzy Logic

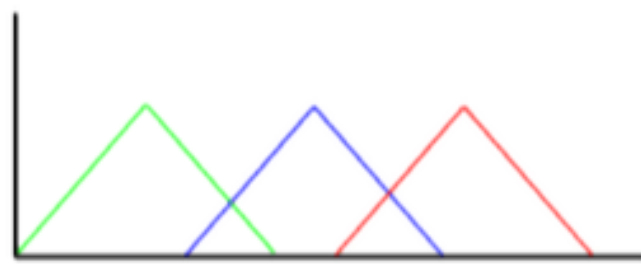


Fuzzy Logic

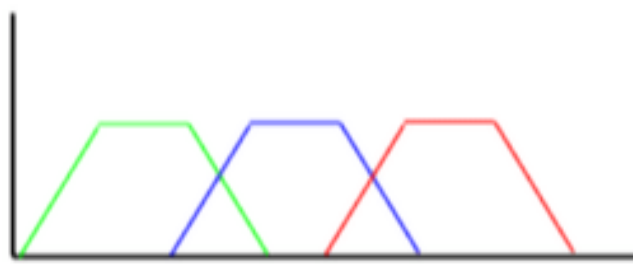
- Graphs are very important
 - Shape of the graph is called the Fuzzy Manifold
 - Each classification 'young' and 'old' is called the linguistic variable.
 - The useful fuzzy operations or 'fuzzy control' is determined by combining the fuzzy manifolds of different linguistic variables.

Membership Functions

- Membership functions determine the fuzzy manifold of a membership
- Lots of options beyond the ones I've used



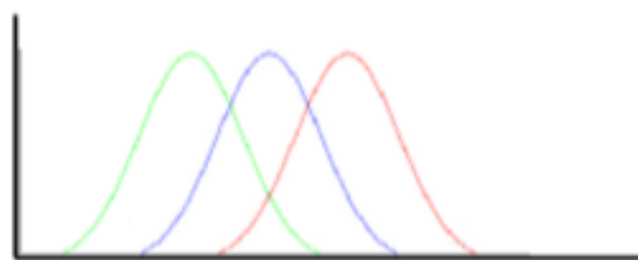
triangular



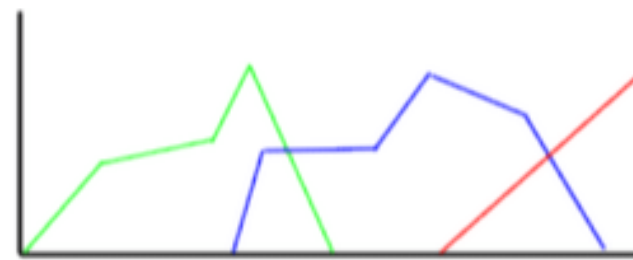
trapezoidal



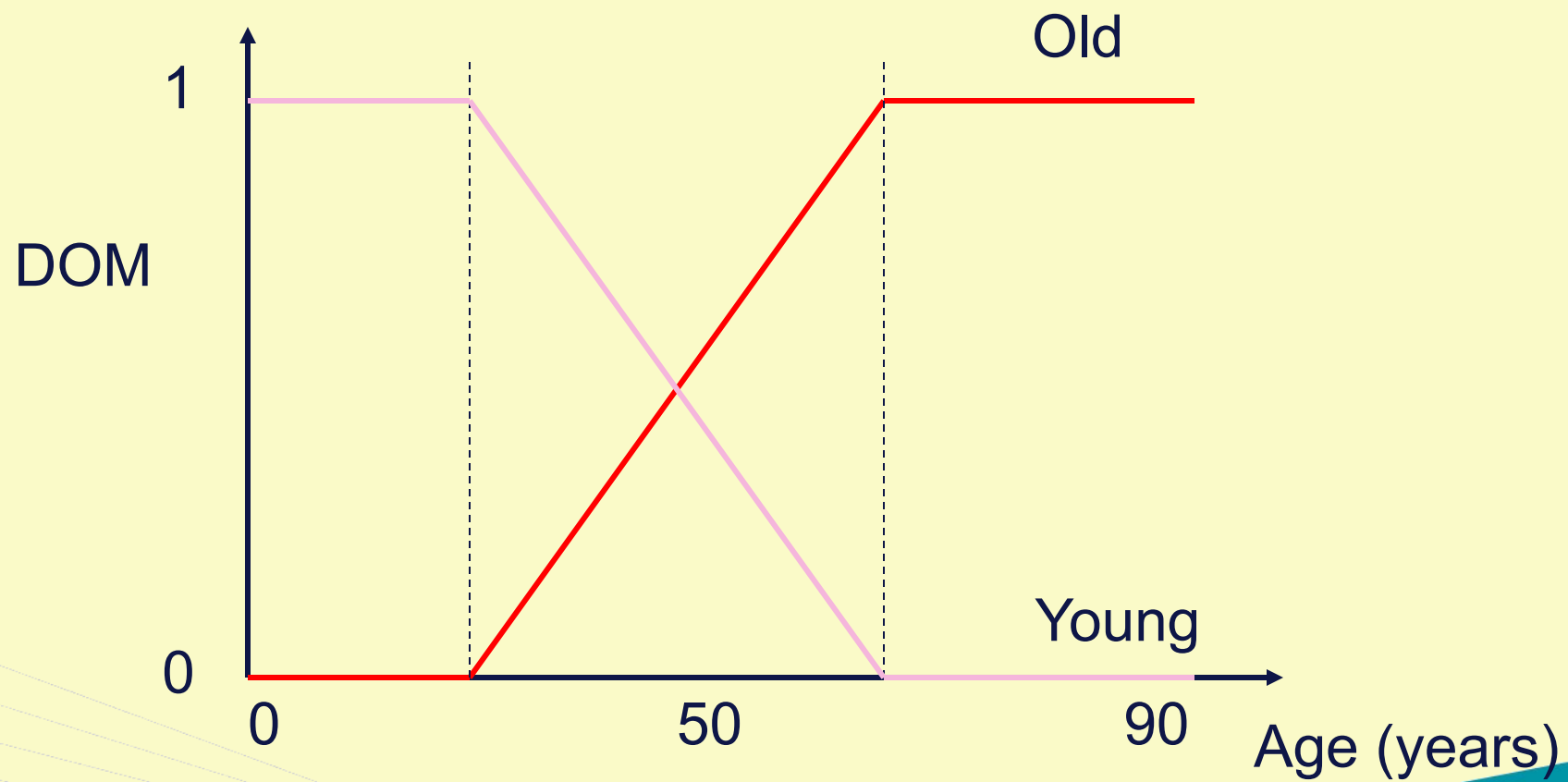
singleton



Gaussian



Piecewise linear





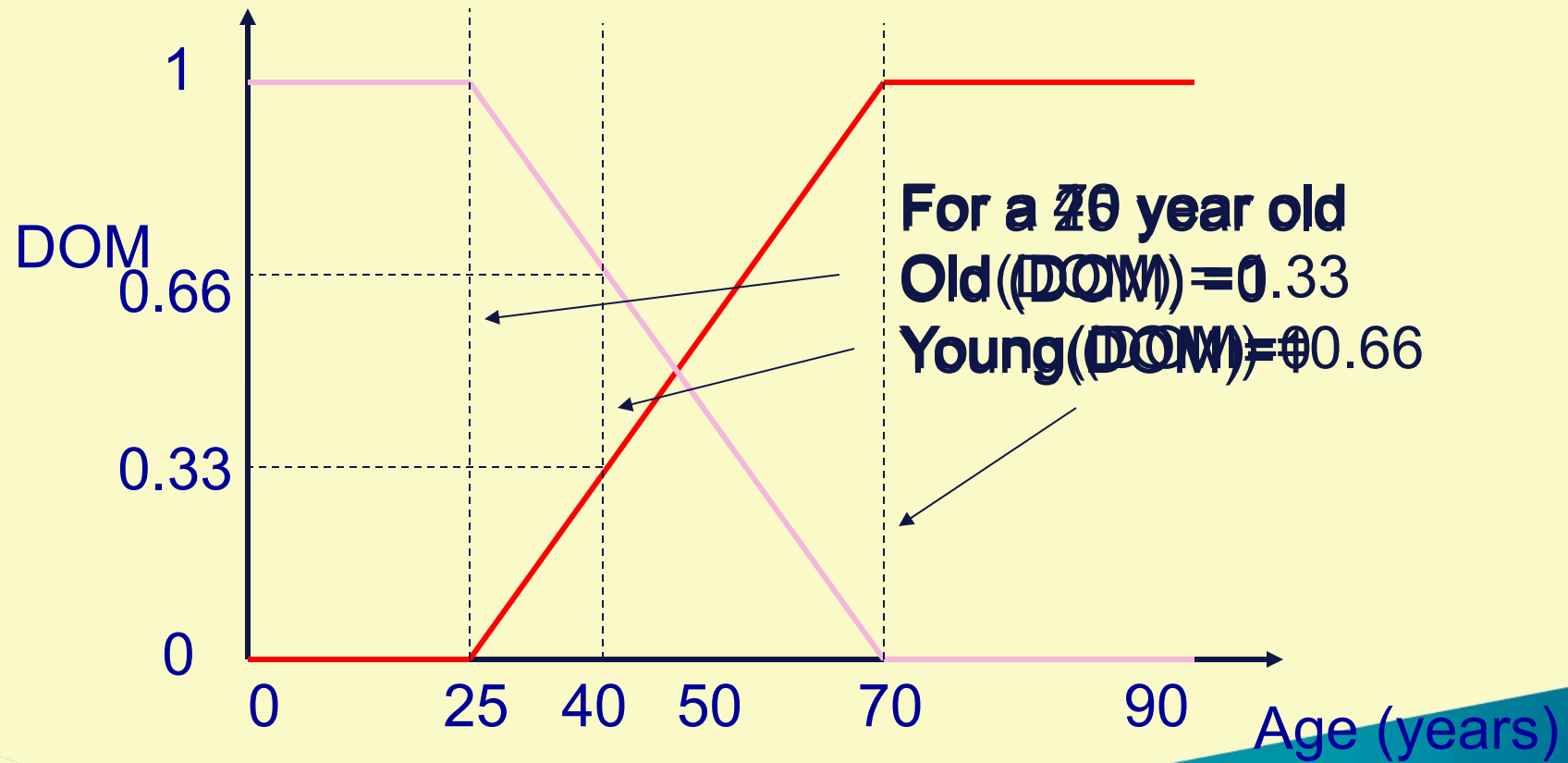
UNIVERSITY
OF HULL

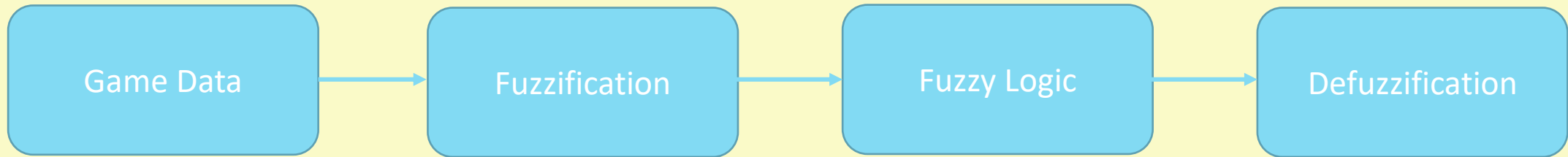
Fuzzy Logic

- The individual Degree Of Membership can now be classified for a given age
- This will give us a fuzzy output for the age
- This is done for a given age by drawing a line vertically from the age and recording where it crosses each fuzzy manifold.



Fuzzy Logic





- Crisp game data
- Fuzzified into a series of sets of degrees of membership
- Perform fuzzy logic operations



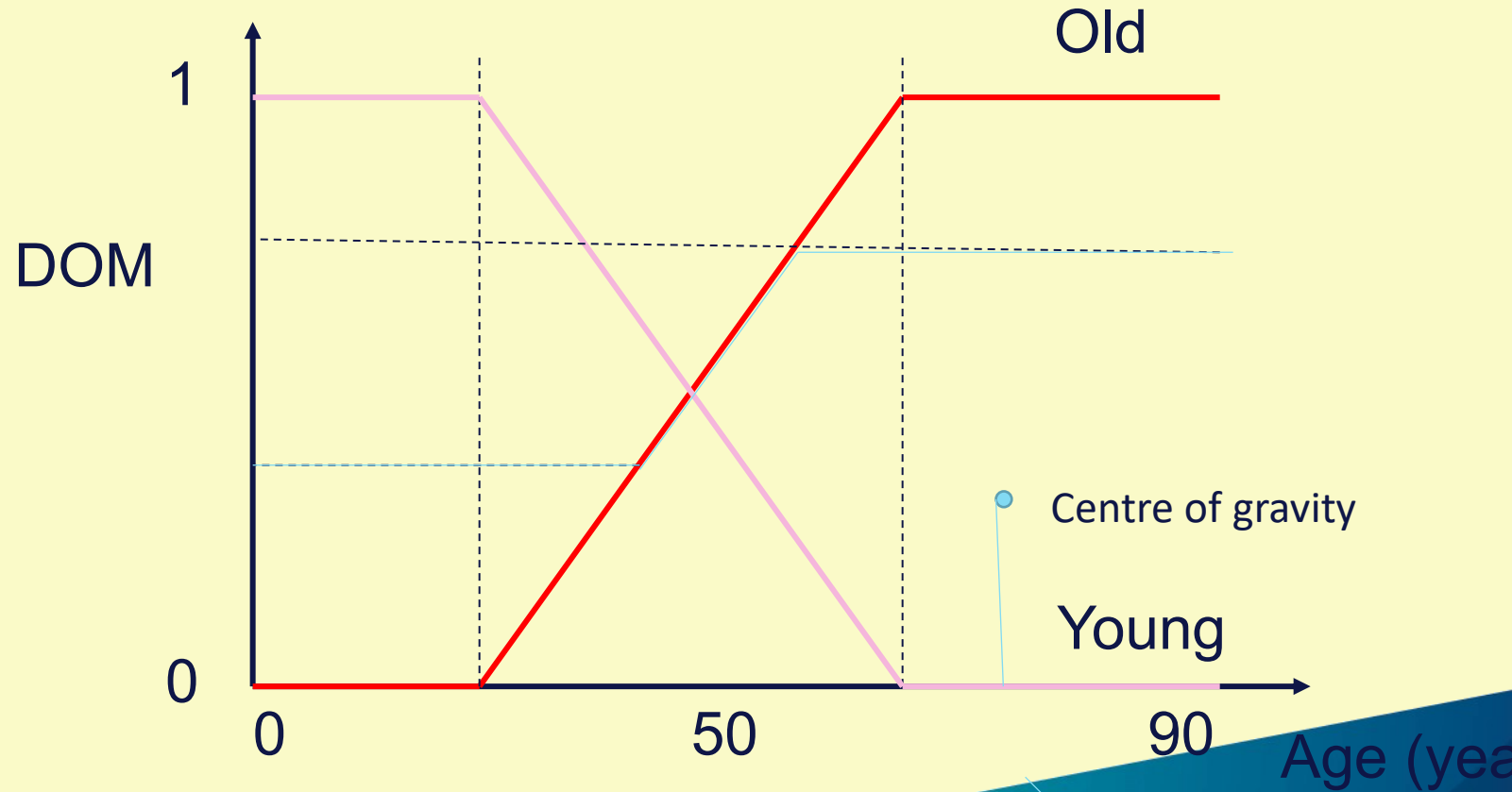
Fuzzy Logic

- Now we can return to our set theory and define a fuzzy set for each age
 - For 25 Fuzzy set = $\{1,0\}$
 - For 70 Fuzzy set = $\{0,1\}$
 - For 40 Fuzzy set = $\{0.66,0.33\}$
- How do we use these fuzzy sets to get a real output from the system?
 - How do we *defuzzify* the output?



Defuzzification methods

- Highest membership
 - Select the set with the highest membership
- Blending by membership
 - If the output is a scalable value – for example speed of movement then can blend the walk speed with the run speed according to the membership of each
 - Need to ensure that the multiplication factors are normalised
- Centre of gravity of area





One Method: Fuzzy Matrix

- A fuzzy matrix is the equivalent of a load of IF-THEN-ELSEs
- If ***antecedent*** Then ***consequent***
- For each antecedent calculate the membership of those sets to determine a conclusion value
- Combine all conclusion values into fuzzy set
- Defuzzify to determining action

Fuzzy Matrix

- Here there are just 2 dimensions, but you can have many more

Hit Points/Fire Power	Very low HP	Low HP	Medium HP	High HP	Very High HP
Very Weak FP	Retreat!	Retreat!	Defend	Defend	Attack
Weak FP	Retreat!	Defend	Defend	Attack	Attack
Medium FP	Retreat!	Defend	Attack	Attack	Full Attack!
High FP	Retreat!	Defend	Attack	Attack	Full Attack!
Very High FP	Defend	Attack	Attack	Full Attack!	PWN!

When to use fuzzy logic

- Fuzzy logic can be used *anywhere* where there are conditional
- For example, the transitions in state machines are typically controlled using conditional
- These could be made “fuzzy”

Hit Points/Fire Power	Very low HP	Low HP	Medium HP	High HP	Very High HP
Very Weak FP	Retreat!	Retreat!	Defend	Defend	Attack
Weak FP	Retreat!	Defend	Defend	Attack	Attack
Medium FP	Retreat!	Defend	Attack	Attack	Full Attack!
High FP	Retreat!	Defend	Attack	Attack	Full Attack!
Very High FP	Defend	Attack	Attack	Full Attack!	PWN!